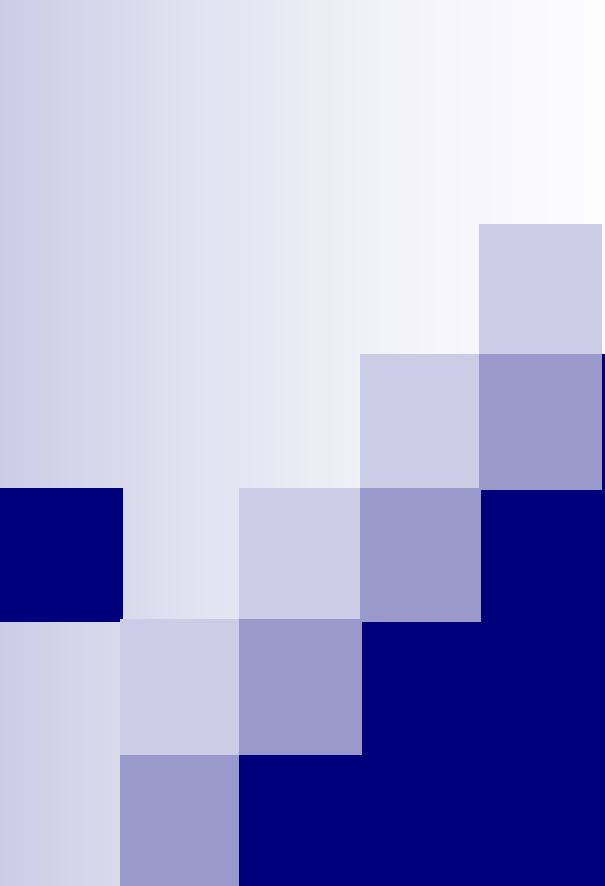




数据结构

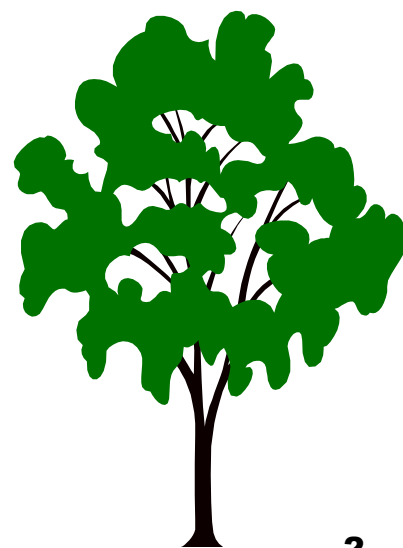
第六章 树和二叉树

主讲：陈锦秀

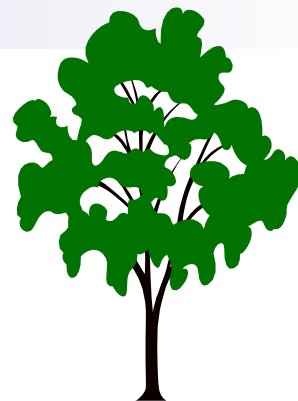
厦门大学信息学院计算机系

第六章 树和二叉树

- 6.1 树的定义和基本概念
- 6.2 二叉树
 - 6.2.1 树的定义和基本术语
 - 6.2.2 二叉树的性质
 - 6.2.3 二叉树的存储结构
- 6.3 遍历二叉树
 - 6.3.1 遍历二叉树
 - 6.3.2 线索二叉树
- 6.4 树和森林
 - 6.4.1 树的存储结构
 - 6.4.2 森林与二叉树的转换
 - 6.4.3 树和森林的遍历
- 6.6 赫夫曼树及其应用
 - 6.6.1 最优二叉树（赫夫曼树）
 - 6.6.2 赫夫曼编码
- 6.7 回溯法与树的遍历



6.1 树的定义和基本术语

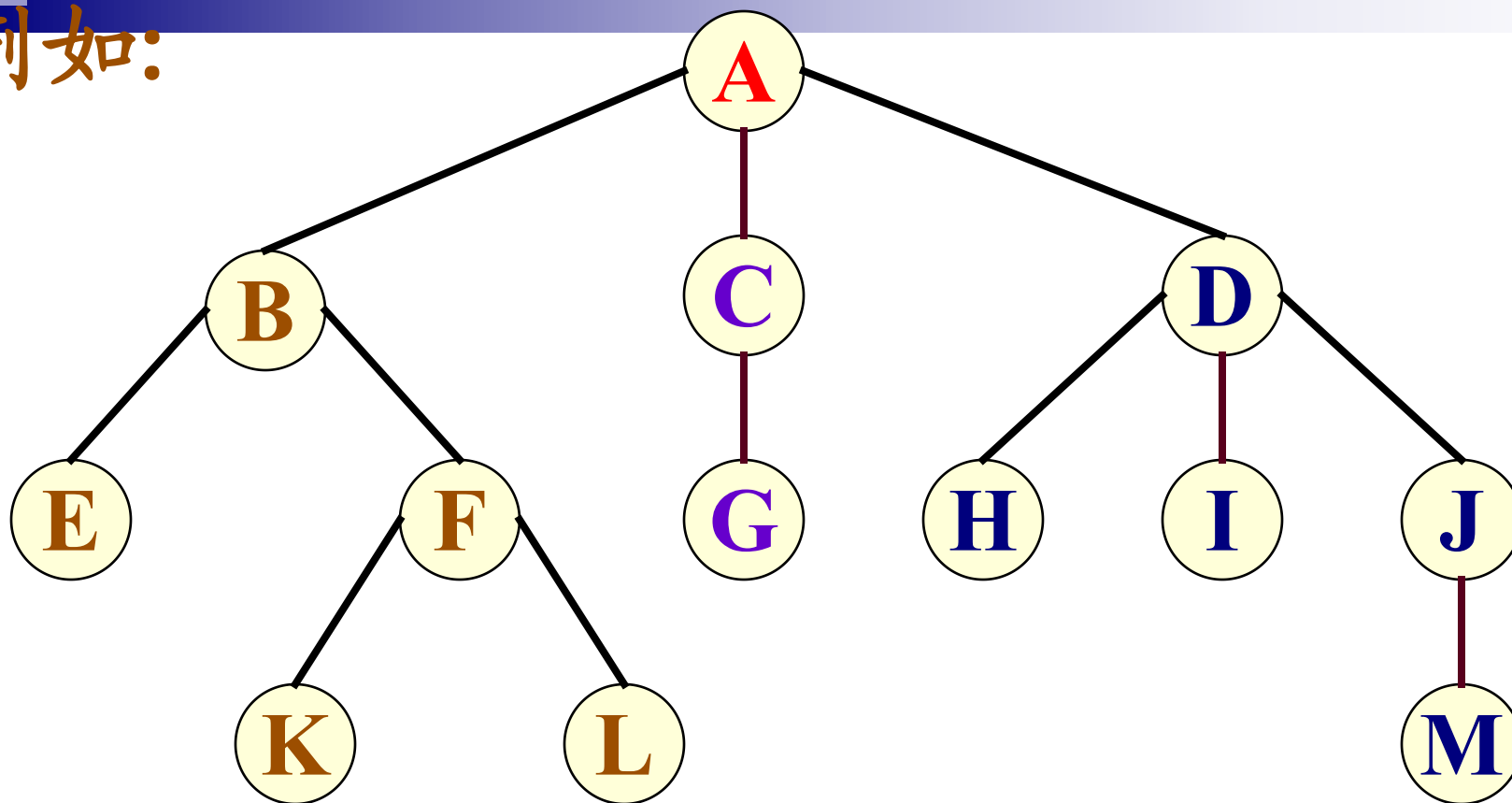


- 树型结构是一类重要的**非线性结构**。
 - 树型结构是结点之间有分支，并且具有**层次关系**的结构，它非常类似于自然界中的树。
 - 树结构在客观世界里是大量存在的，例如家谱、行政组织机构都可用树形象地表示。
 - 树在计算机领域中也有着广泛的应用，例如在**编译程序**中，用树来表示源程序的语法结构；在**数据库系统**中，可用树来组织信息；在**分析算法的行为**时，可用树来描述其执行过程等等。

6.1 树的定义和基本术语

- 定义：树(Tree)是 $n(n \geq 0)$ 个结点的有限集 T ， T 为空时称为空树，否则它满足如下两个条件：
 - (1) 有且仅有一个特定的称为根(Root)的结点；
 - (2) 其余的结点可分为 $m(m \geq 0)$ 个互不相交的子集 $T_1, T_2, T_3 \dots T_m$ ，其中每个子集又是一棵树，并称其为子树(Subtree)。
- 树还有其他的表示形式，如图6.2的三种表示法：
 - (1) 嵌套集合
 - (2) 广义表的形式
 - (3) 凹入表示法

例如:



A(**B**(**E**, **F**(**K**, **L**)), **C**(**G**), **D**(**H**, **I**, **J**(**M**)))

树根 T_1 T_2 T_3

对比树型结构和线性结构 的结构特点



线性结构

第一个数据元素
(无前驱)

最后一个数据元素
(无后继)

其它数据元素
(一个前驱、
一个后继)

树型结构

根结点
(无前驱)

多个叶子结点
(无后继)

其它数据元素
(一个前驱、
多个后继)

6.1 树的定义和基本术语

有向树：

(1) 有确定的根；

(2) 树根和子树根之间为有向关系。

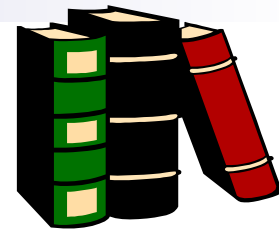
有序树：

子树之间存在确定的次序关系。

无序树：

子树之间不存在确定的次序关系。

基本术语:



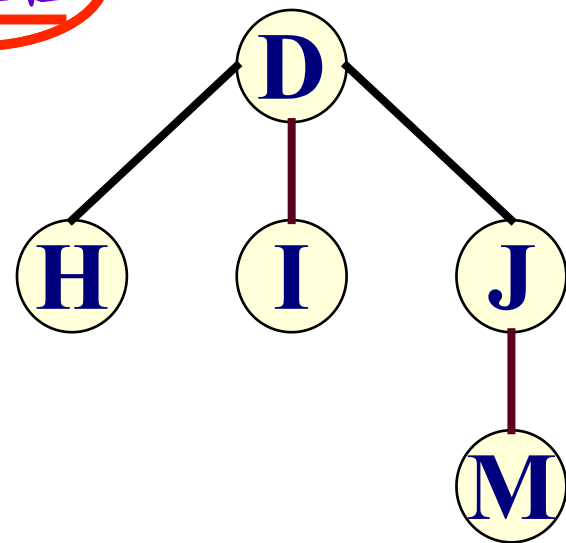
结点: 数据元素+若干指向子树的分支

结点的度: 结点拥有的子树(分支的个数)

树的度: 树中所有结点的度的最大值

叶子结点: 度为零的结点, 其余结点称为非终端结点或分支结点。

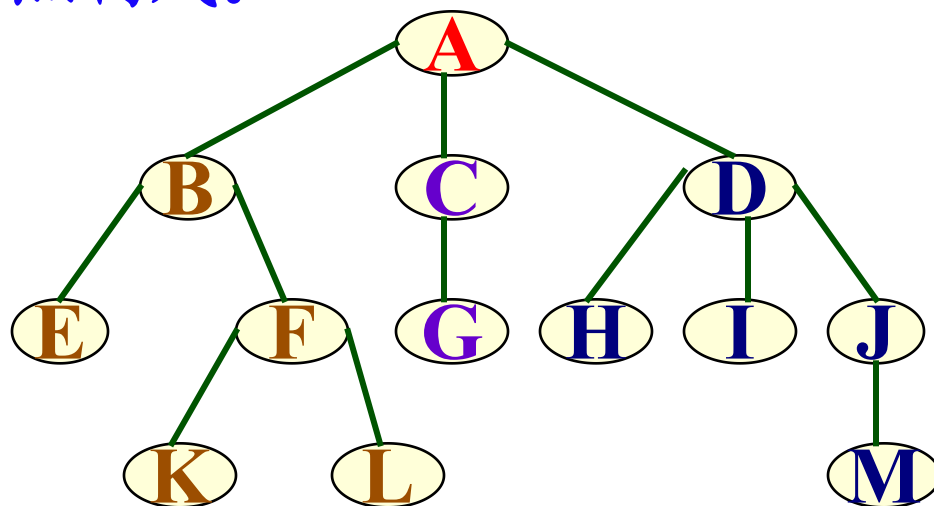
分支结点: 度大于零的结点



(从根到结点的)路径: 由从根到该结点所经分支和结点构成。

孩子结点:

结点的子树的根称为该结点的孩子。相应地, 该结点称为**双亲**。同一个双亲的孩子称为**兄弟**。依此可以递归定义**祖先**和**子孙**的概念。

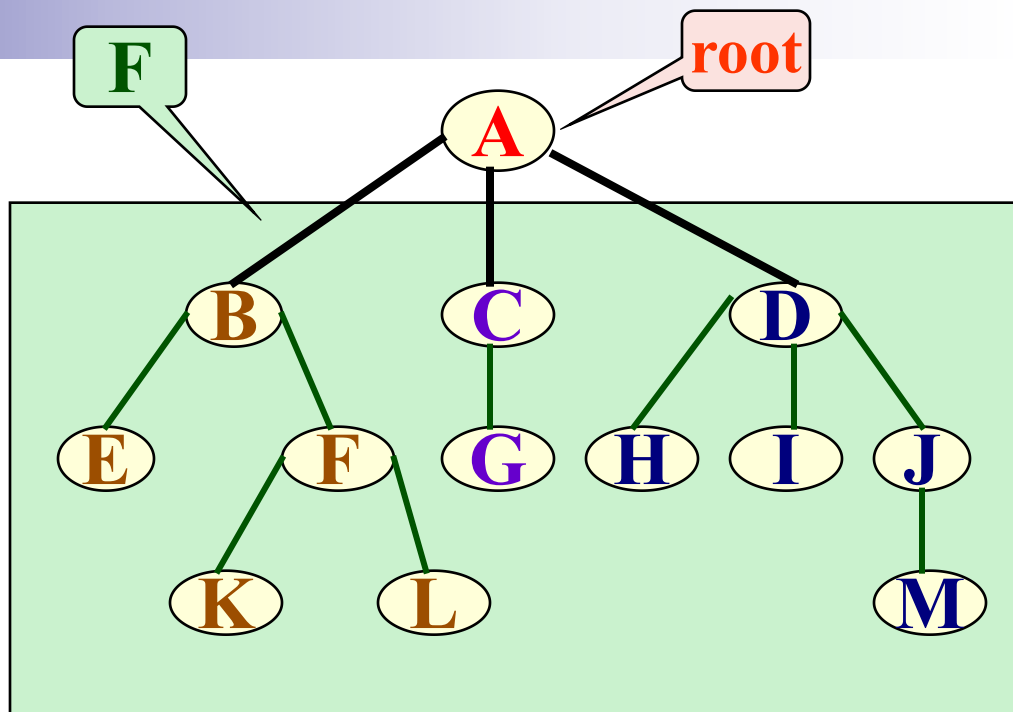


结点的层次: 从根开始定义, 根为第1层, 根的孩子为第2层。若某个结点在第 l 层, 则其子树的根就在第 $l+1$ 层。双亲在同一层的结点互为堂兄弟。

树的深度: 树中叶子结点所在的最大层次。

森林:

- 是 m ($m \geq 0$) 棵互不相交的树的集合。
 - 对树中每个结点而言，其子树的集合即为森林。
- 由此，可以用森林和树相互递归的定义来描述树：



任何一棵非空树是一个二元组

$\text{Tree} = (\text{root}, F)$

其中：root 被称为根结点，F 被称为子树森林

$F = (T_1, T_2, T_3 \dots T_m)$, T_i 称做根root的第*i*棵子树。

6.1 树的定义和基本术语

基本操作:

查 找 类

插 入 类

删 除 类

查找类:

Root(T) // 求树的根结点

Value(T, cur_e) // 求当前结点的元素值

Parent(T, cur_e) // 求当前结点的双亲结点

LeftChild(T, cur_e) // 求当前结点的最左孩子

RightSibling(T, cur_e) // 求当前结点的右兄弟

TreeEmpty(T) // 判定树是否为空树

TreeDepth(T) // 求树的深度

TraverseTree(T, Visit()) // 遍历

插入类:

InitTree(&T) // 初始化置空树

CreateTree(&T, definition)

// 按定义构造树

Assign(T, cur_e, value)

// 给当前结点赋值

InsertChild(&T, &p, i, c)

// 将以c为根的子树插入为结点p的第i棵子树

删除类:

ClearTree(&T) // 将树清空

DestroyTree(&T) // 销毁树的结构

DeleteChild(&T, &p, i)

// 删除结点p的第i棵子树

第六章 树和二叉树

6.1 树的定义和基本概念

6.2 二叉树

6.2.1 树的定义和基本术语

6.2.2 二叉树的性质

6.2.3 二叉树的存储结构

6.3 遍历二叉树

6.3.1 遍历二叉树

6.3.2 线索二叉树

6.4 树和森林

6.4.1 树的存储结构

6.4.2 森林与二叉树的转换

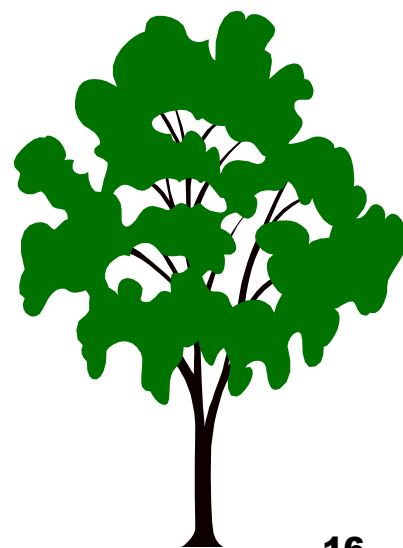
6.4.3 树和森林的遍历

6.6 赫夫曼树及其应用

6.6.1 最优二叉树（赫夫曼树）

6.6.2 赫夫曼编码

6.7 回溯法与树的遍历



6.2 二叉树

- 二叉树在树结构的应用中起着非常重要的作用，因为
 1. 对二叉树的许多操作算法简单，
 2. 而任何树都可以与二叉树相互转换，这样就解决了树的存储结构及其运算中存在的复杂性。



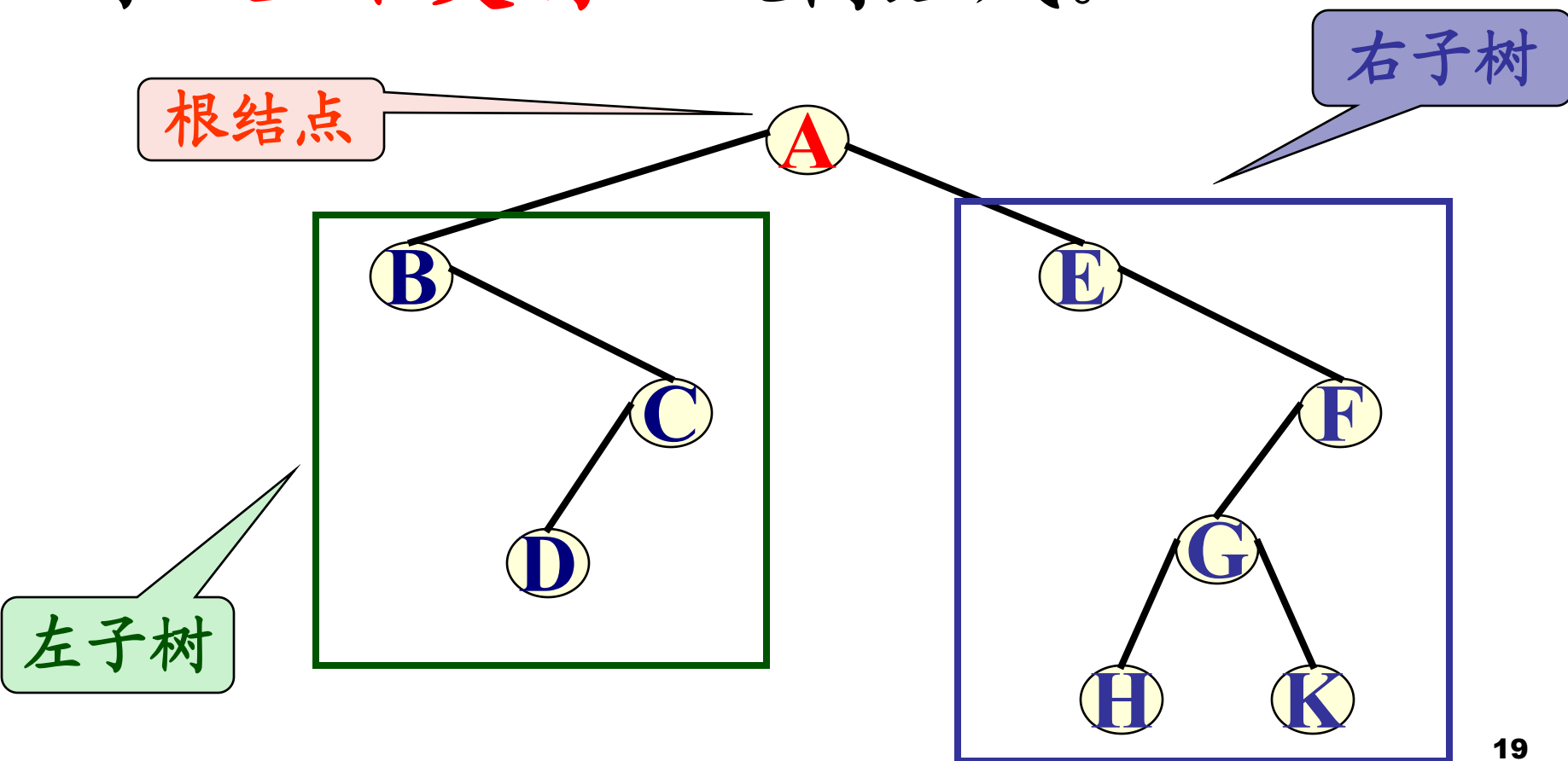
6.2.1 二叉树的定义

定义：二叉树是由 $n(n \geq 0)$ 个结点的有限集合构成，此集合或者为空集，或者由一个根结点及至多两棵互不相交的左右子树组成，并且左右子树都是二叉树，且次序不能任意颠倒。

■ 这也是一个递归定义。二叉树可以是空集合，根可以有空的左子树或空的右子树。二叉树不是树的特殊情况，它们是两个概念。

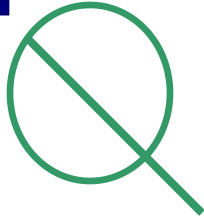
■ 二叉树结点的子树要区分左子树和右子树，即使只有一棵子树也要进行区分，说明它是左子树，还是右子树。这是二叉树与树的最主要的差别（图6.8 (C) 和图6.8 (d) 是不同的两棵二叉树。）。

二叉树或为空树，或是由一个根结点加上两棵分别称为**左子树**和**右子树**的、**互不交的**二叉树组成。



二叉树的五种基本形态:

空树

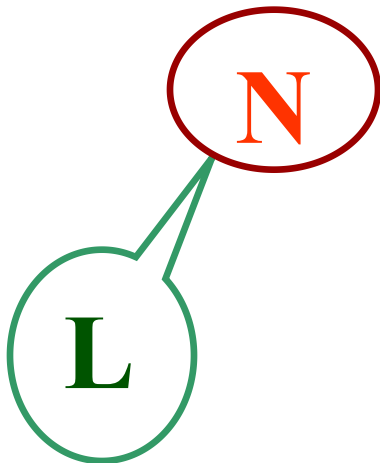


只含根结点

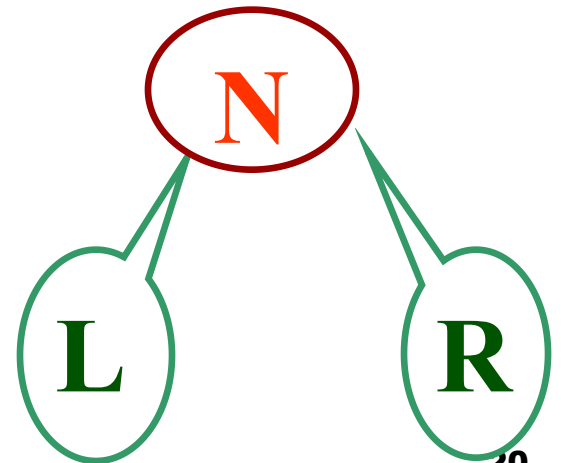
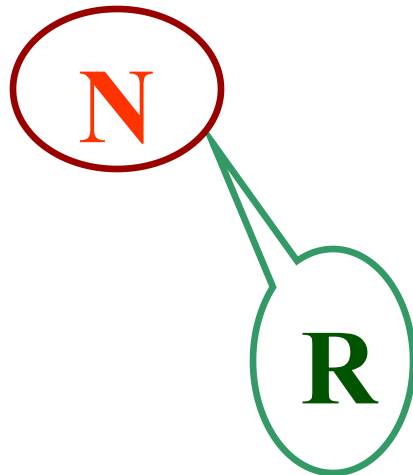


左右子
树均不
为空树

右子树为空树



左子树为空树



6.2.1 二叉树的定义

二叉树的主要基本操作：

查 找 类

插 入 类

删 除 类

6.2.2 二叉树的性质

性质1: 在二叉树的第 i 层上至多有 2^{i-1} 个结点($i \geq 1$)。

■ 用归纳法证明:

- **归纳基:** 当 $i=1$ 时, 只有一个根结点, $2^{i-1}=2^0=1$, 命题成立。
- **归纳假设:** 假定对于所有的 j , $1 \leq j < i$, 命题成立, 即第 j 层上至多有 2^{j-1} 个结点。由归纳假设可知, 第 $i-1$ 层上至多有 2^{i-2} 个结点。
- **归纳证明:** 二叉树上每个结点至多有两棵子树, 则第 i 层的结点数 $= 2 \times 2^{i-2} = 2^{i-1}$ 。

6.2.2 二叉树的性质

性质2: 深度为 k 的二叉树至多有 $2^k - 1$ 个结点
($k \geq 1$)。

■ 证明:

深度为 k 的二叉树的最大的结点数为二叉树中每层上的最大结点数之和, 由性质1得到每层上的最大结点数:

$$\begin{aligned} E_{i=1}^k \text{ (第 } i \text{ 层上的最大结点数)} &= E_{i=1}^k 2^{i-1} \\ &= 2^0 + 2^1 + \dots + 2^{k-1} = 2^k - 1 \end{aligned}$$

6.2.2 二叉树的性质

性质3: 对任何一棵二叉树, 如果其终端叶子结点数为 n_0 , 度为2的结点数为 n_2 , 则 $n_0 = n_2 + 1$ 。

■ 证明:

- 设二叉树中度为1的结点数为 n_1 , 二叉树中总结点数为 N , 因为二叉树中所有结点均小于或等于2, 所以有:

$$N = n_0 + n_1 + n_2 \quad (6-1)$$

- 再看二叉树中的分支数, 除根结点外, 其余结点都有一个进入分支, 设 B 为二叉树中的分支总数, 则有: $N = B + 1$ 。
- 由于这些分支都是由度为1和2的结点射出的, 所以有:

$$B = n_1 + 2 * n_2$$

$$N = B + 1 = n_1 + 2 * n_2 + 1 \quad (6-2)$$

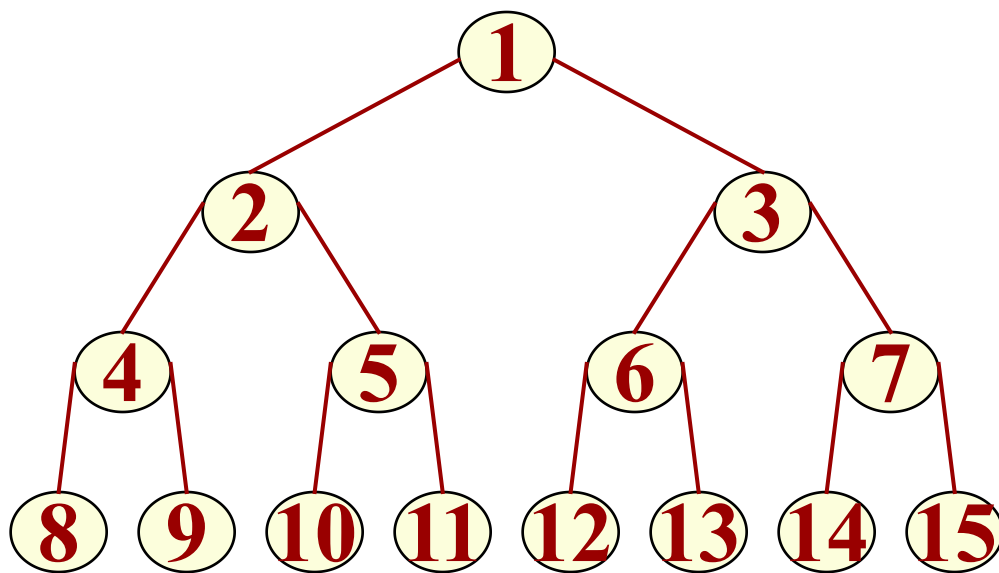
- 由式 (6-1) 和 (6-2) 得到:

$$n_0 + n_1 + n_2 = n_1 + 2 * n_2 + 1$$

$$n_0 = n_2 + 1$$

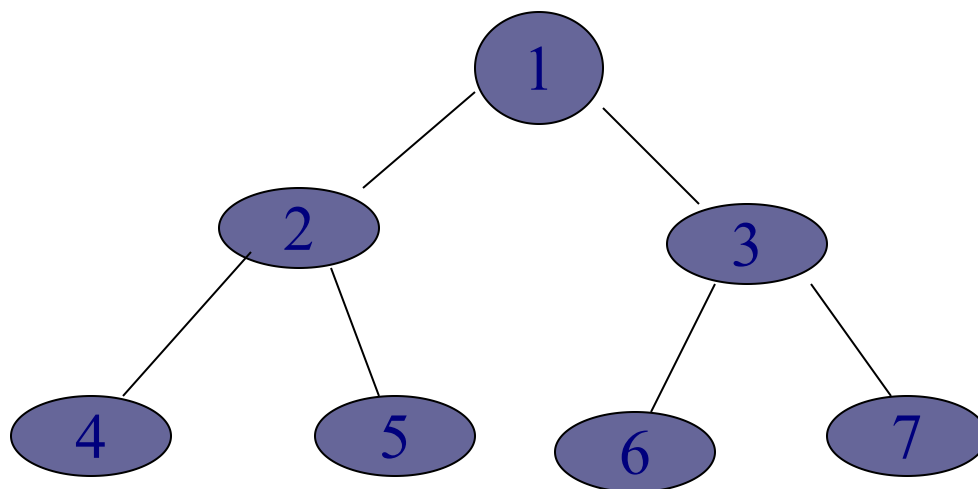
6.2.2 二叉树的性质

- 下面介绍两种特殊形态的二叉树：**满二叉树**和**完全二叉树**。
- 一棵深度为 k 且有 2^k-1 个结点的二叉树称为**满二叉树**。
下图就是一棵满二叉树，对结点进行了顺序编号。

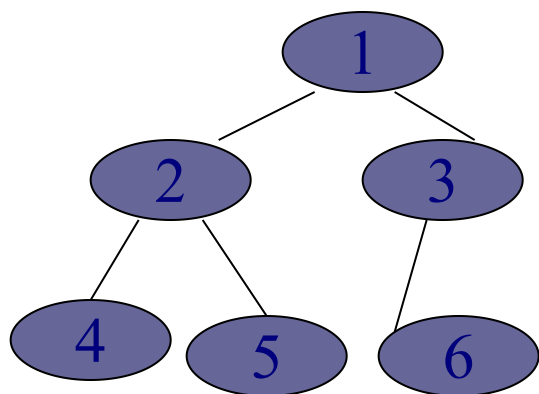


6.2.2 二叉树的性质

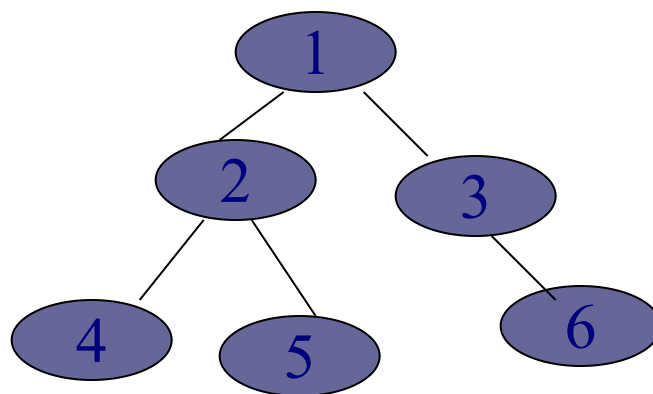
- 如果深度为 k 、由 n 个结点的二叉树中各结点能够与深度为 k 的顺序编号的满二叉树从1到 n 标号的结点相对应，则称这样的二叉树为完全二叉树。
- 满二叉树是完全二叉树的特例。
- 完全二叉树的特点是：
 - (1) 所有的叶结点都出现在第 k 层或 $k-1$ 层。
 - (2) 任一结点，如果其右子树的最大层次为 k ，则其左子树的最大层次为 k 或 $k+1$ 。



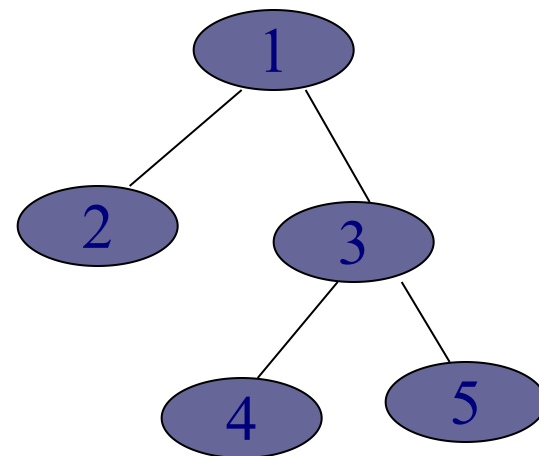
满二叉树



(a) 完全二叉树



(b) 非完全二叉树



(c) 非完全二叉树

6.2.2 二叉树的性质

性质4: 具有 n 个结点的完全二叉树的深度为 $\lfloor \log_2 n \rfloor + 1$ 。符号 $\lfloor x \rfloor$ 表示不大于 x 的最大整数。

■ 假设此二叉树的深度为 k ，则根据性质2及完全二叉树的定义得到： $2^{k-1} - 1 < n \leq 2^k - 1$ 或

$$2^{k-1} \leq n < 2^k$$

取对数得到： $k - 1 \leq \log_2 n < k$

因为 k 是整数，所以有： $k = \lfloor \log_2 n \rfloor + 1$ 。

6.2.2 二叉树的性质

性质5: 如果对一棵有 n 个结点的完全二叉树的结点按层序编号（从第1层到第 $\lfloor \log_2 n \rfloor + 1$ 层，每层从左到右），则对任一结点 i ($1 \leq i \leq n$), 有:

- (1) 如果 $i=1$ ，则结点 i 无双亲，是二叉树的根；如果 $i>1$ ，则其双亲是结点 $\lfloor i/2 \rfloor$ 。
- (2) 如果 $2i>n$ ，则结点 i 为叶子结点，无左孩子；否则，其左孩子是结点 $2i$ 。
- (3) 如果 $2i+1>n$ ，则结点 i 无右孩子；否则，其右孩子是结点 $2i+1$ 。

我们只要先证明 (2) 和 (3)，便可以导出 (1)。

■ 以下使用数学归纳法证明 (2) 和 (3)。

基础步：对于 $i=1$ ，由完全二叉树的定义，其左孩子（若有）编号为结点2，其右孩子（若有）编号为结点3。

归纳假设步：假设结点 i 满足命题，去证明结点 $i+1$ 也满足命题。

对于 $i>1$ ，可分为两种情况：

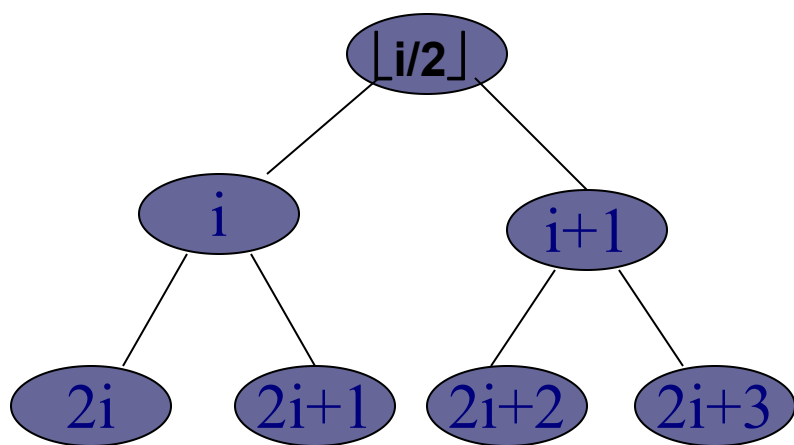
a. 设 i 和 $i+1$ 结点在同一层，则 i 的左右孩子分别为 $2i$ 和 $2i+1$ ，那么， $i+1$ 结点的左右孩子分别为 $2i+2$ 和 $2i+3$ ，也就是 $2(i+1)$ 和 $2(i+1)+1$ 。

b. 如果 i 和 $i+1$ 结点不在同一层，则 i 的左右孩子分别为 $2i$ 和 $2i+1$ ，那么， $i+1$ 结点的左右孩子分别为 $2i+2$ 和 $2i+3$ ，也就是 $2(i+1)$ 和 $2(i+1)+1$ ，符合定义。

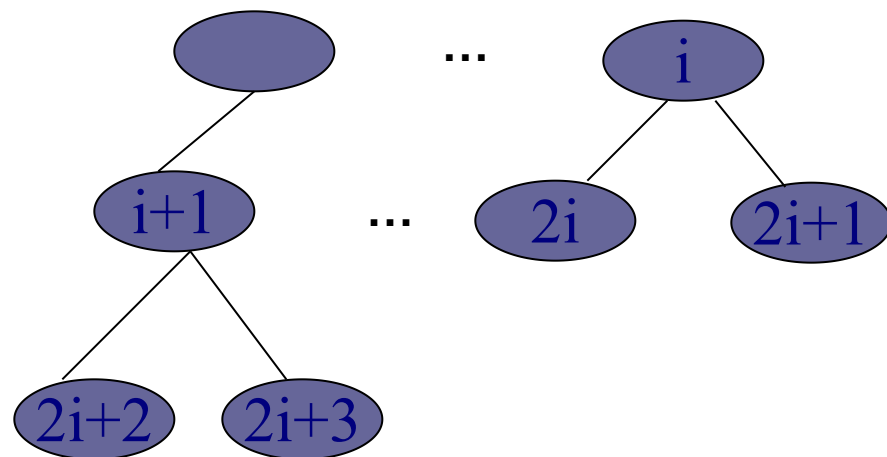
(2) (3) 证明完毕。

6.2.2 二叉树的性质

如图所示为完全二叉树上结点及其左右孩子在结点之间的关系。



(a) i 和 $i+1$ 结点在同一层



(b) i 和 $i+1$ 结点不在同一层

6.2.2 二叉树的性质

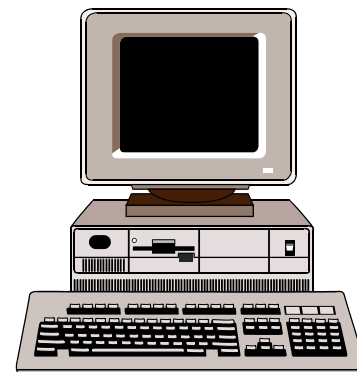
证明性质5中的（1）：

- 当 $i=1$ 时，就是根，因此无双亲，
- 当 $i>1$ 时，
 - 如果 i 为左孩子，因为 $2 \times (i/2) = i$ ，则 $i/2$ 是 i 的双亲；
 - 如果 i 为右孩子， $i = 2p + 1$ ， i 的双亲应为 p ， $p = (i - 1) / 2 = \lfloor i/2 \rfloor$ 。
- 证毕。

6.2.3 二叉树的存储结构

1. 二叉树的顺序存储表示

2. 二叉树的链式存储表示



6.2.3 二叉树的存储结构——顺序存储结构

■ **顺序存储结构**：它是用一组连续的存储单元存储二叉树的数据元素。

□ 因此，必须把二叉树的所有结点安排成为一个恰当的序列，结点在这个序列中的相互位置能反映出结点之间的**逻辑关系**。

□ **用编号的方法**：从树根起，自上层至下层，每层自左至右的给所有结点编号。

6.2.3 二叉树的存储结构——顺序存储结构

```
#define MAX_TREE_SIZE 100
```

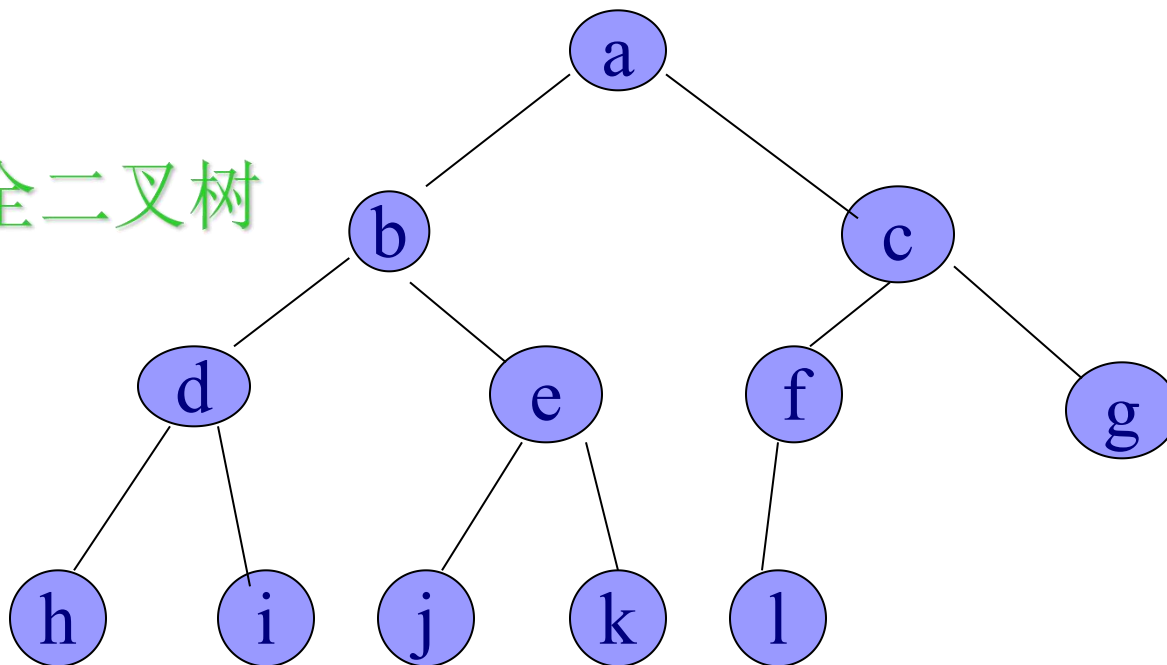
```
// 二叉树的最大结点数
```

```
typedef TElemType SqBiTree[MAX_TREE_SIZE];
```

```
// 0号单元存储根结点
```

```
SqBiTree bt;
```

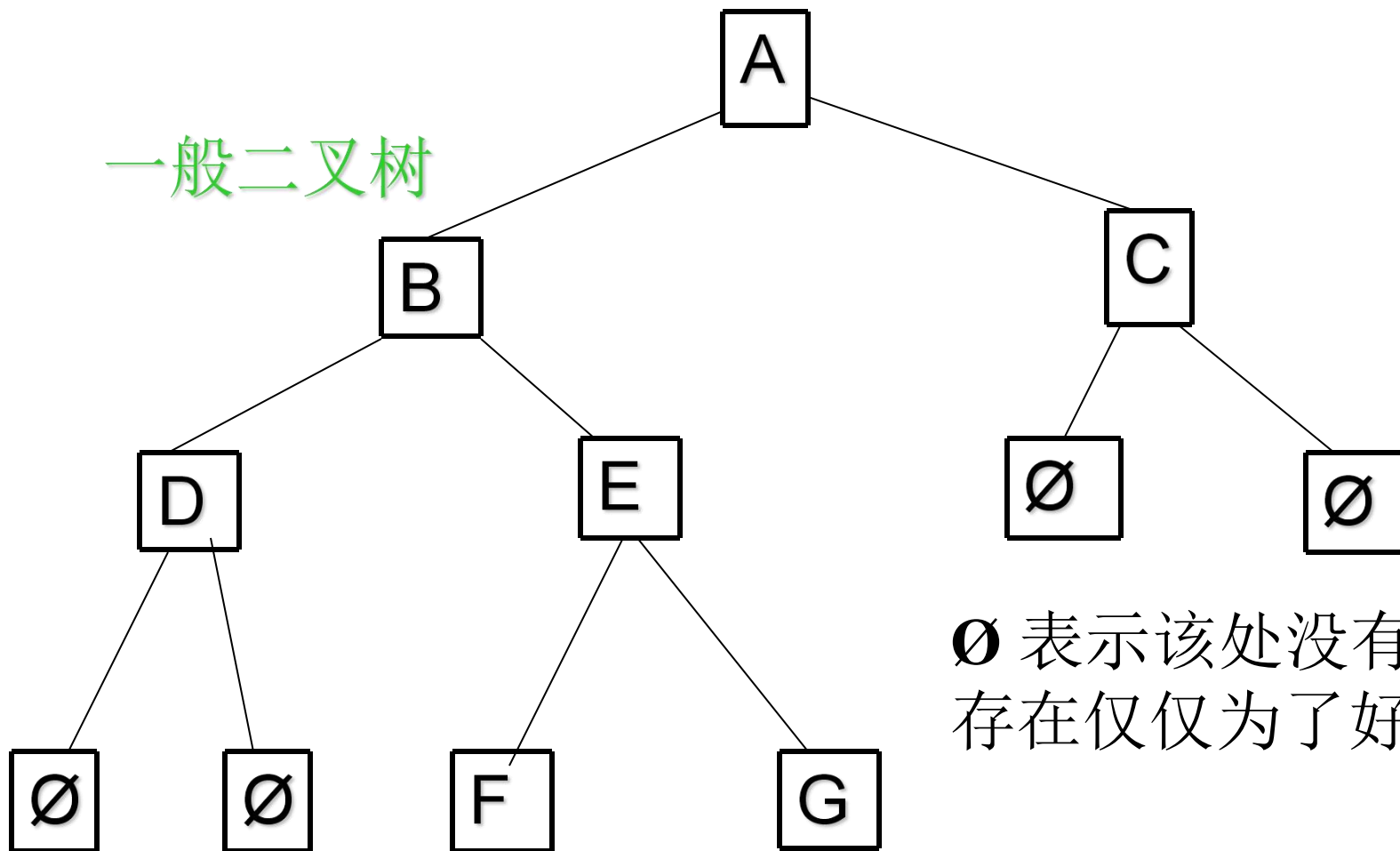

完全二叉树



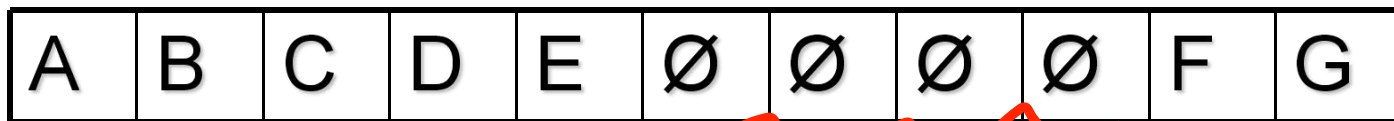
1 2 3 4 5 6 7 8 9 10 11 12

A	B	C	D	E	F	G	H	I	J	K	L
---	---	---	---	---	---	---	---	---	---	---	---

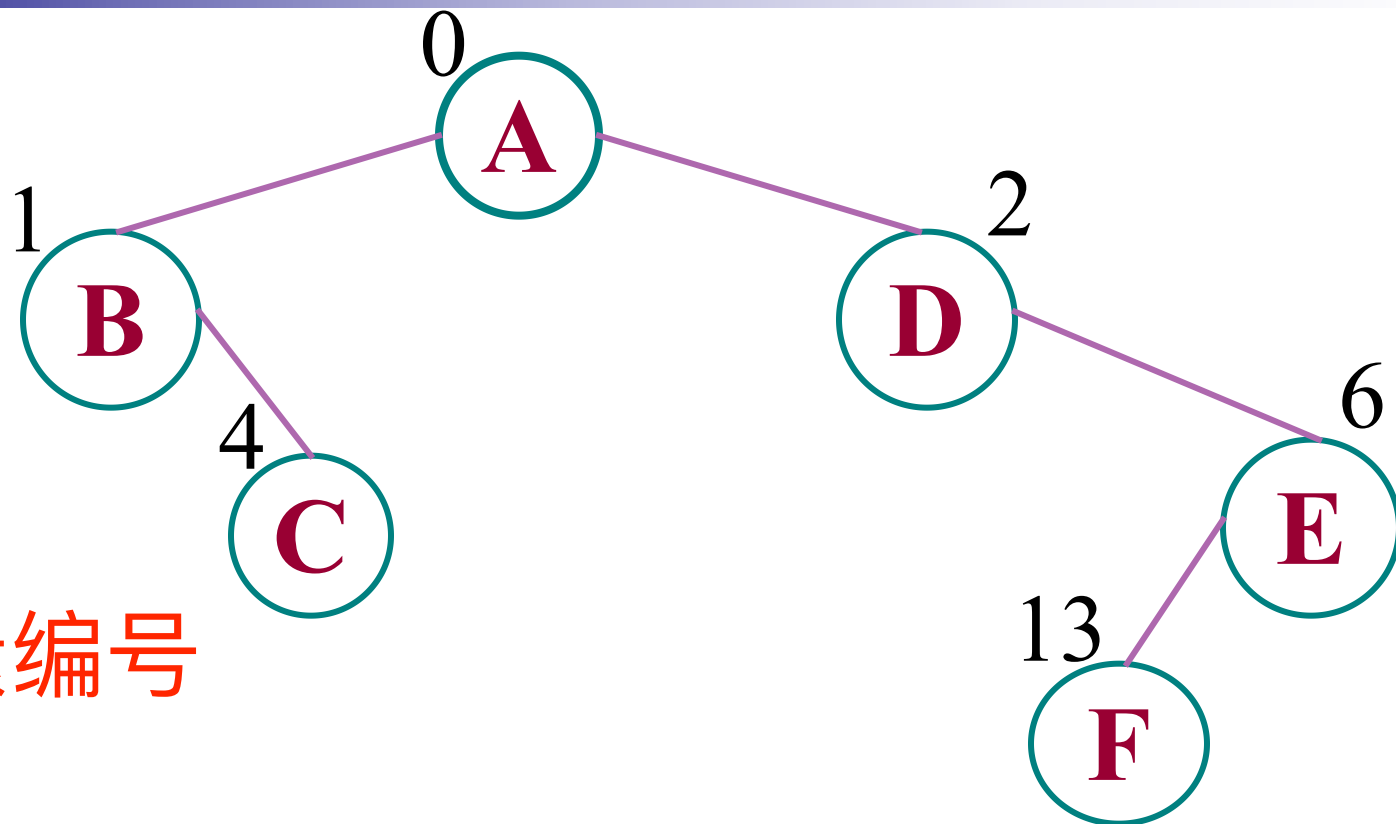
一般二叉树



∅ 表示该处没有元素
存在仅仅为了好理解



例如:



先给元素编号

0	1	2	3	4	5	6	7	8	9	10	11	12	13
A	B	D		C		E							F

空元素

6.2.3 二叉树的存储结构——顺序存储结构

■ 缺点:

- 有可能对存储空间造成极大的浪费，在最坏的情况下，一个深度为 h 且只有 h 个结点的右单支树却需要 2^h-1 个结点存储空间。
- 而且，若经常需要插入与删除树中结点时，顺序存储方式不是很好！

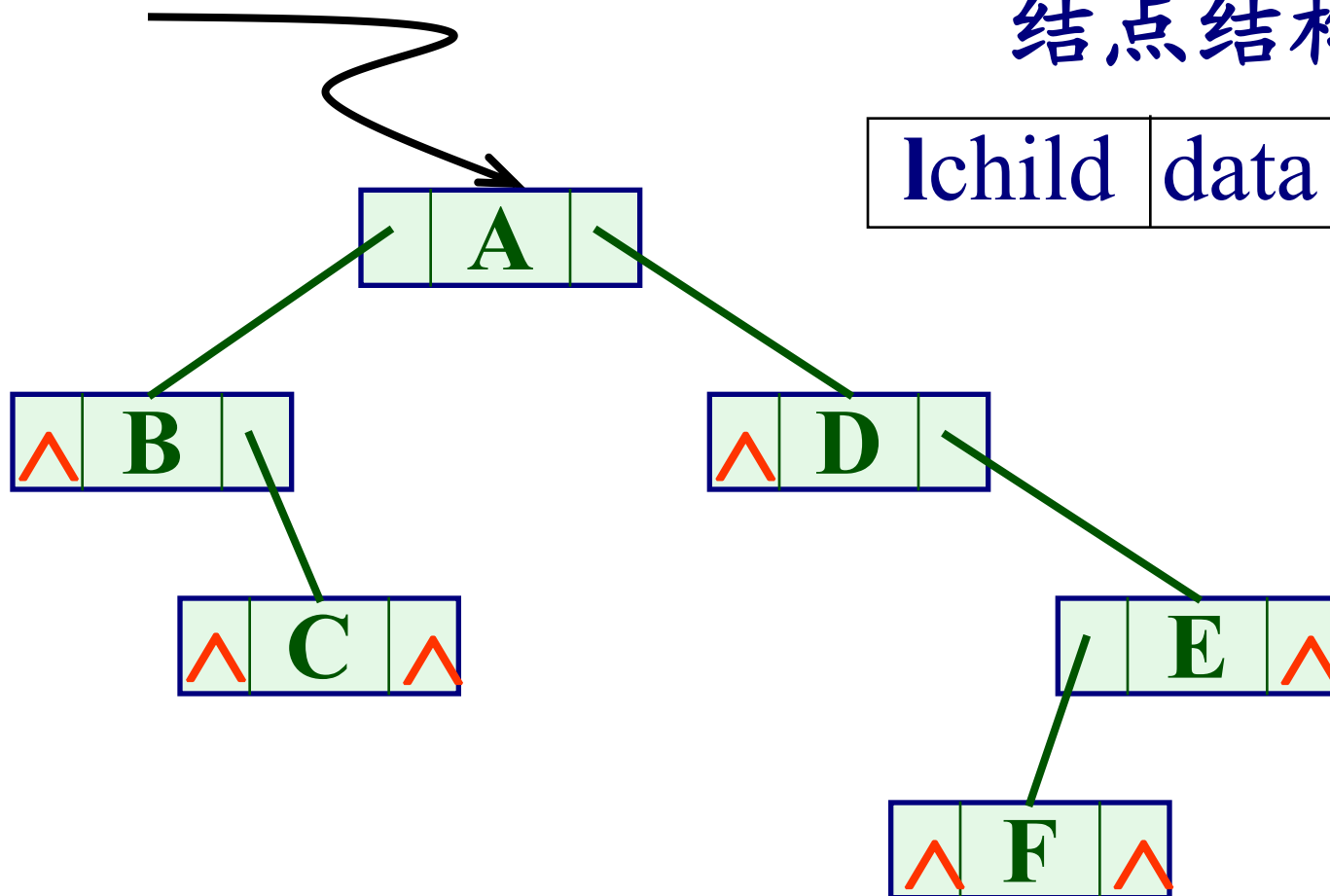
6.2.3 二叉树的存储结构——链式存储结构

- 存储二叉树经常用**二叉链表法**

root

结点结构:

lchild	data	rchild
--------	------	--------



二叉链表C 语言的类型描述如下:

```
typedef struct BiTNode { // 结点结构
    TElemType    data;
    struct BiTNode *lchild, *rchild;
                        // 左右孩子指针
} BiTNode, *BiTree;
```

结点结构:



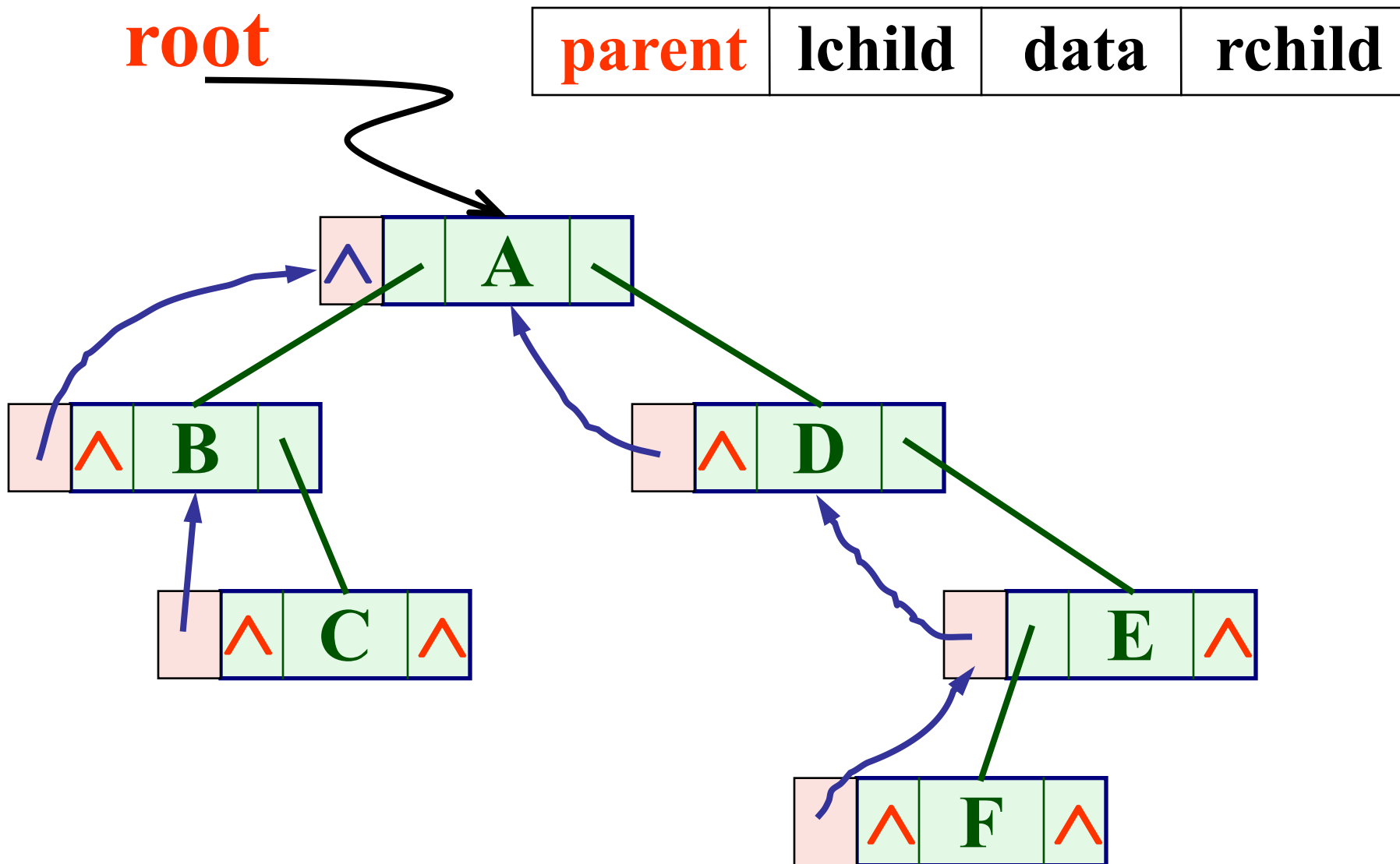
6.2.3 二叉树的存储结构——链式存储结构

- 二叉链表中，有时也可用数组的下标来模拟指针，即开辟三个一维数组data, lchild, rchild 分别存储结点的元素及其左, 右指针域；
- 此外，为了便于找到结点的双亲，还可在结点结构中增加一个指向其双亲结点的指针域，这种结构称为三叉链表，如下图。

lchild	data	parent	rchild
--------	------	--------	--------

三叉链表:

结点结构:



三叉链表C 语言的类型描述如下:

```
typedef struct TriTNode { // 结点结构
    TElemType    data;
    struct TriTNode *lchild, *rchild;
                        // 左右孩子指针
    struct TriTNode *parent; //双亲指针
} TriTNode, *TriTree;
```

结点结构:

parent	lchild	data	rchild
--------	--------	------	--------

第六章 树和二叉树

6.1 树的定义和基本概念

6.2 二叉树

6.2.1 树的定义和基本术语

6.2.2 二叉树的性质

6.2.3 二叉树的存储结构

6.3 遍历二叉树

6.3.1 遍历二叉树

6.3.2 线索二叉树

6.4 树和森林

6.4.1 树的存储结构

6.4.2 森林与二叉树的转换

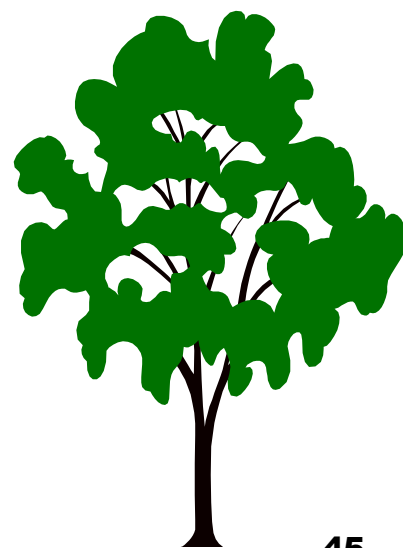
6.4.3 树和森林的遍历

6.6 赫夫曼树及其应用

6.6.1 最优二叉树（赫夫曼树）

6.6.2 赫夫曼编码

6.7 回溯法与树的遍历



6.3 遍历二叉树和线索二叉树

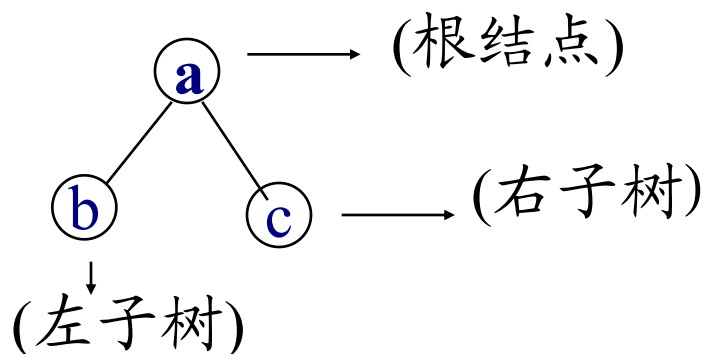
6.3.1 遍历二叉树

■ 问题的提出:

- 在二叉树的一些应用中，常常要求在树中查找具有某种特征的结点，或者对树中全部结点逐一进行某种处理。
- 这就引入了遍历二叉树的问题，即如何按某条搜索路径巡访树中的每一个结点，使得每一个结点均被访问一次，而且仅被访问一次。

6.3.1 遍历二叉树

- “遍历”对线性结构是容易解决的，因为对线性结构而言，只有一条搜索路径(每个结点均只有一个后继)。
- 而二叉树是非线性的，每个结点有两个后继，则存在如何遍历即按什么样的搜索路径遍历的问题。
- 因而需要寻找一种规律，以便使二叉树上的结点能排列在一个线性队列上，从而便于遍历。



由二叉树的递归定义，二叉树的三个基本组成单元是：根结点、左子树和右子树。

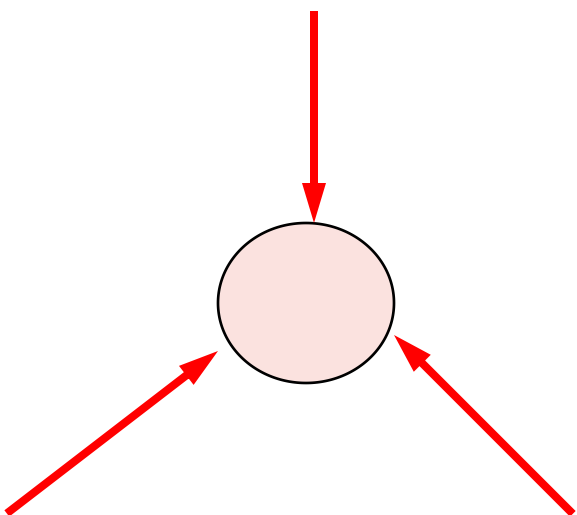
6.3.1 遍历二叉树

对“二叉树”而言，可以有三条搜索路径：

- 1. 先上后下的按层次遍历；
- 2. 先左（子树）后右（子树）的遍历；
- 3. 先右（子树）后左（子树）的遍历。

6.3.1 遍历二叉树

- 假如以L、D、R分别表示遍历左子树、遍历根结点和遍历右子树，遍历整个二叉树则有DLR、LDR、LRD、DRL、RDL、RLD六种遍历方案。
- 若规定先左后右，则只有前三种情况，分别规定为：



DLR——先（根）序的遍历算法

LDR——中（根）序的遍历算法

LRD——后（根）序的遍历算法

先（根）序的遍历算法：

若二叉树为空树，则空操作；否则，

- (1) 访问根结点；
- (2) 先序遍历左子树；
- (3) 先序遍历右子树。



● 中（根）序的遍历算法：

若二叉树为空树，则空操作；否则，

- (1) 中序遍历左子树；
- (2) 访问根结点；
- (3) 中序遍历右子树。



● 后（根）序的遍历算法：

若二叉树为空树，则空操作；否则，

- (1) 后序遍历左子树；
- (2) 后序遍历右子树；
- (3) 访问根结点。

例如下图所示的二叉树表达式

$(a+b*(c-d)-e/f)$

- 若先序遍历此二叉树, 按访问结点的先后次序将结点排列起来, 其先序序列为:

$-+a*b-cd/ef$

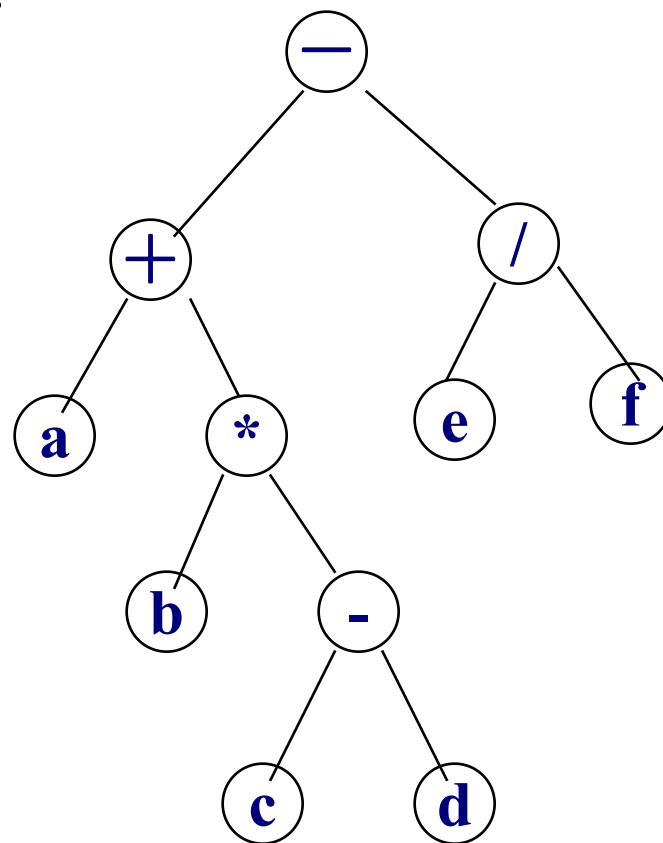
- 按中序遍历, 其中序序列为:

$a+b*c-d-e/f$

- 按后序遍历, 其后序序列为:

$abcd-*+ef/-$

有人喜欢中缀形式的算术表达式, 对于计算机, 使用后缀易于求值。



先序遍历的递归算法，中序与后序与此类似。

```
Status PreOrderTraverse(BiTree T, Status  
    (*Visit)(TElemType e)) {
```

```
//采用二叉链表结构，
```

```
//Visit函数可以是如下打印函数：
```

```
//Status PrintElement(TElemType e)
```

```
//{    printf(e);    return OK; }
```

```
    if (T) {
```

```
        if (Visit(T→data))
```

```
            if (PreOrderTraverse(T→lchild, Visit))
```

```
                if (PreOrderTraverse(T→rchild, Visit))
```

```
                    return OK;
```

```
            return ERROR;
```

```
        } else return OK;
```

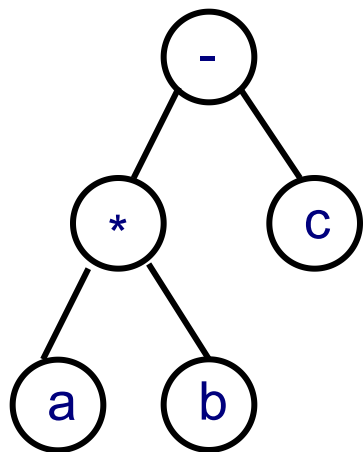
```
    }
```

只要调整一下
Visit() 函数的位置，
就可以简单地转化
为中序遍历算法和
后序遍历算法。

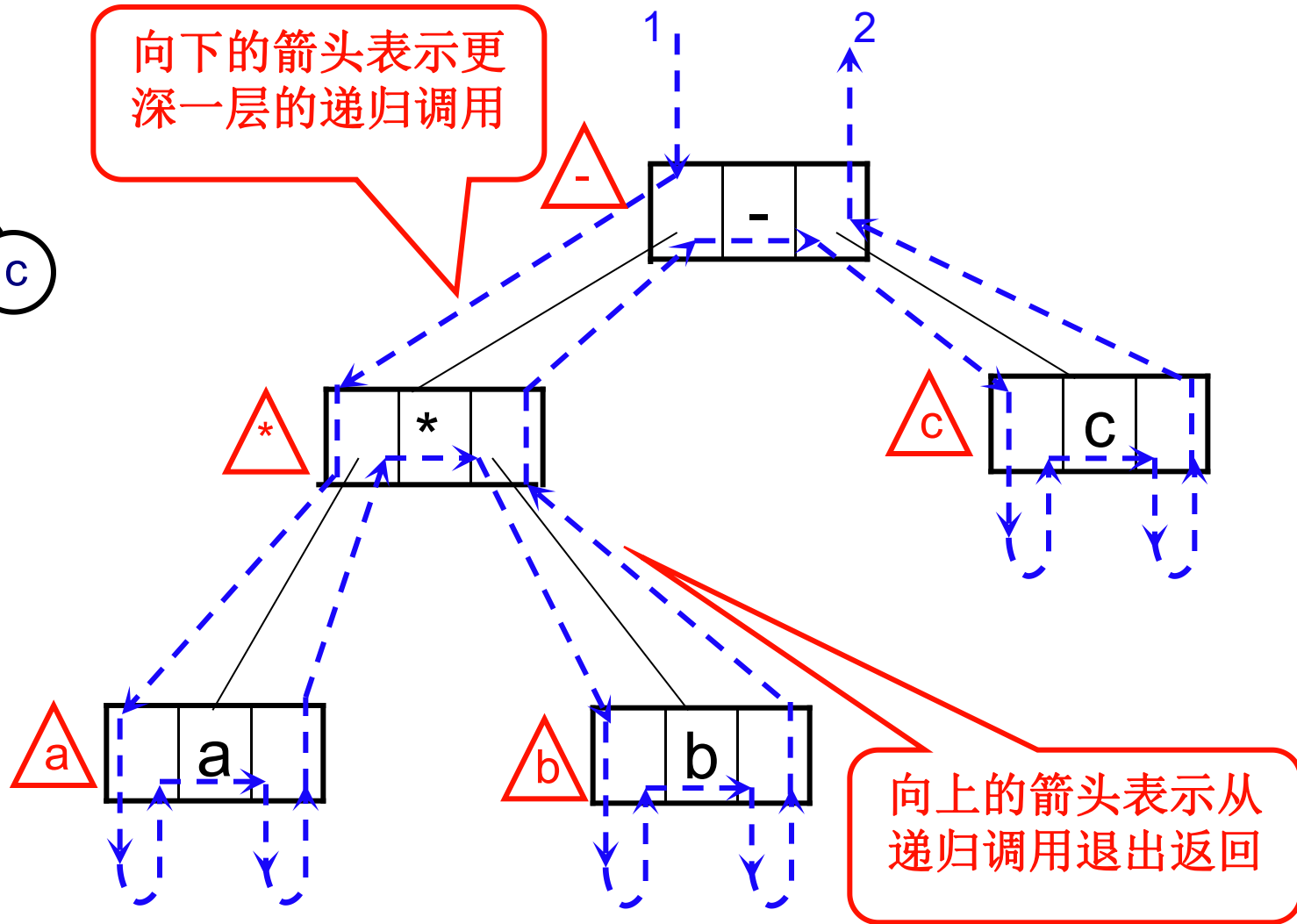
6.3.1 遍历二叉树

- 3种遍历算法之不同处仅在于访问根结点和遍历左、右子树的先后关系。
 - 如果在算法中暂且抹去和递归无关的Visit语句，则3个遍历算法完全相同。
- 如图6.10（b）用带箭头的虚线表示了这3种遍历算法的递归执行过程。
 - 虚线旁的三角形、圆形和方形内的字符分别表示在先序、中序和后序遍历二叉树过程中访问结点时输出的信息。只要沿虚线将沿途所见的三角形（或圆形或方形）内的字符记下，便得遍历二叉树的先序（或中序、或后序）序列。

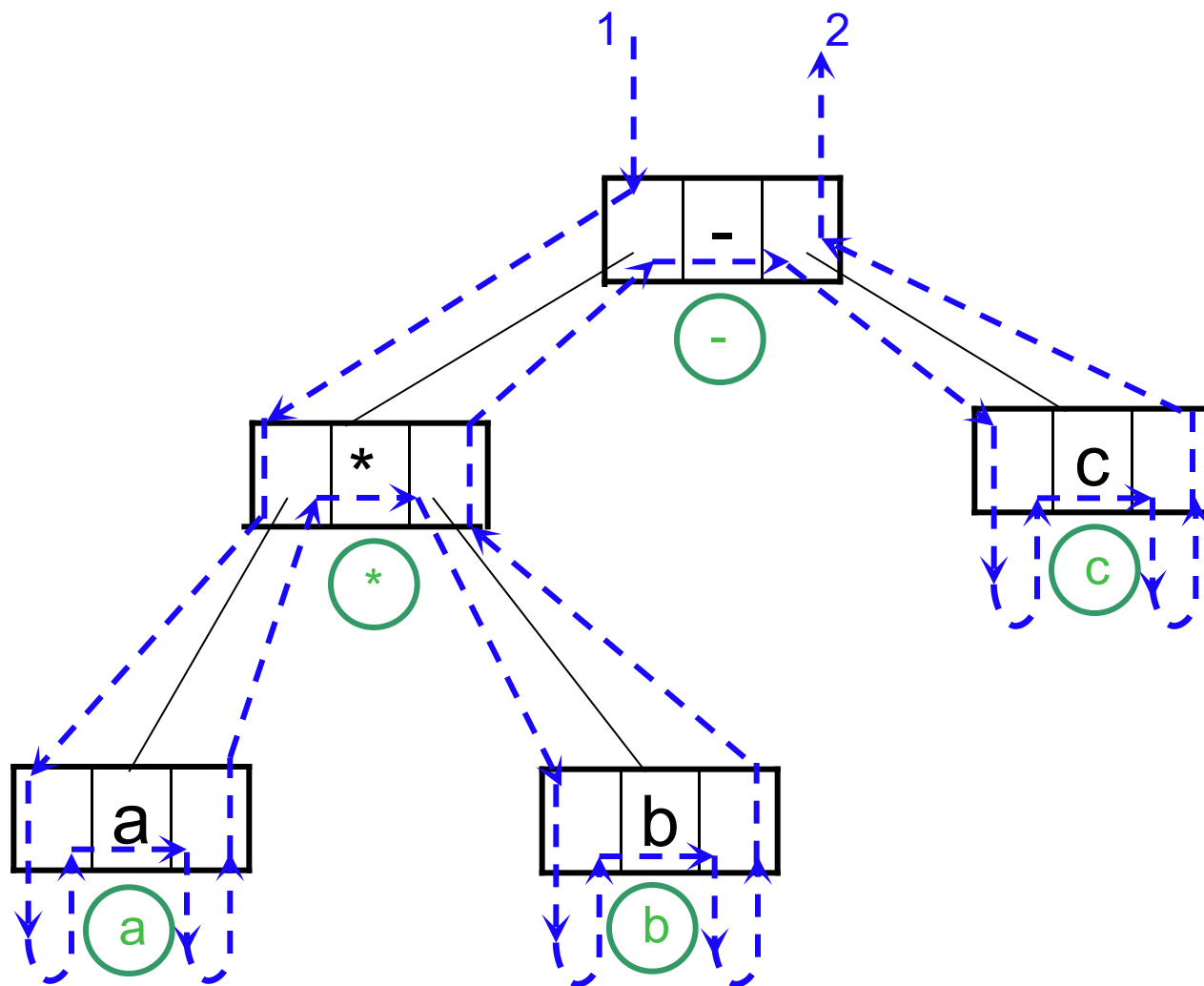
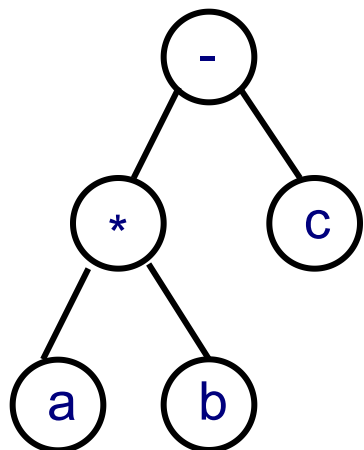
先序遍历二叉树



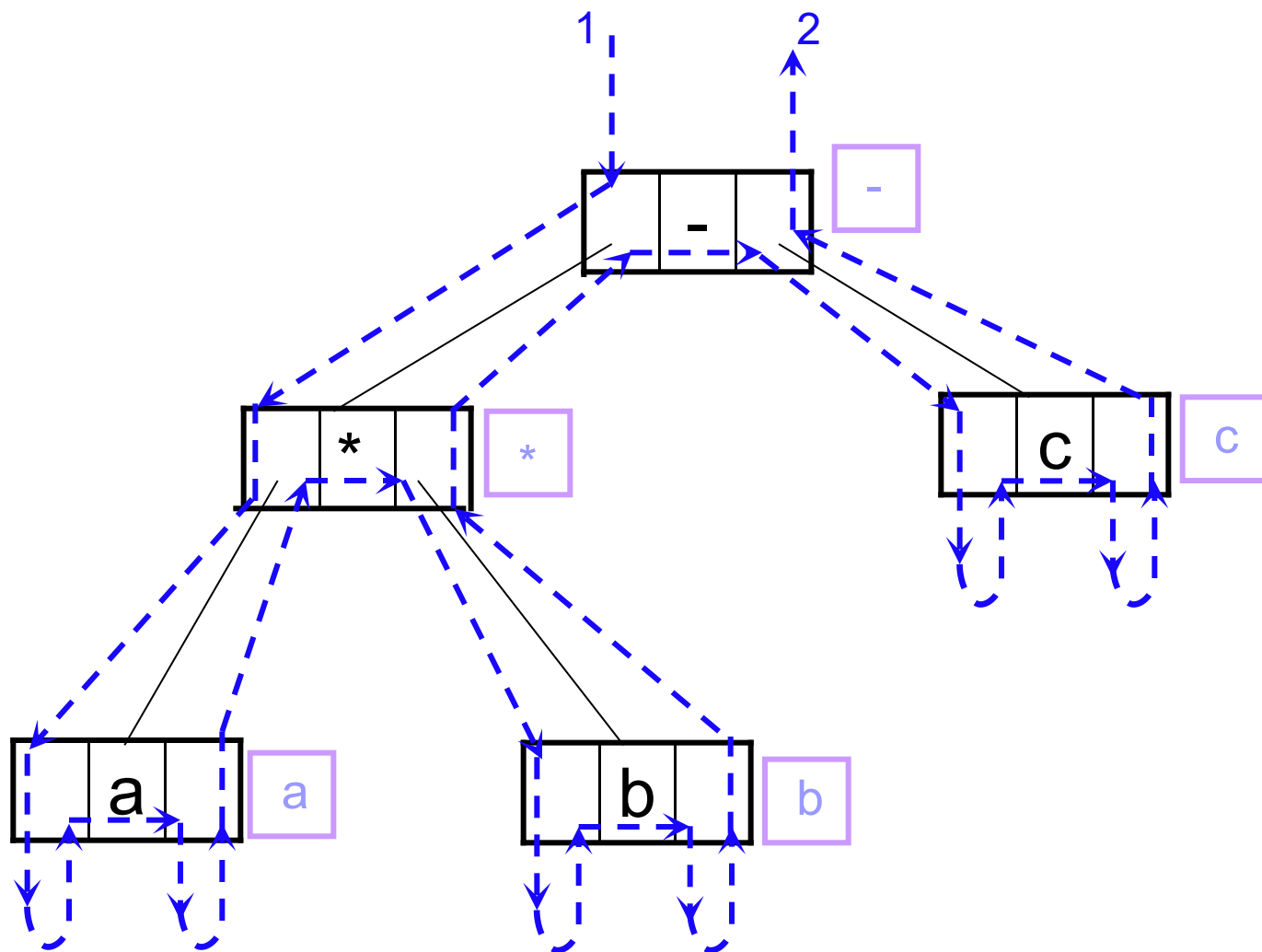
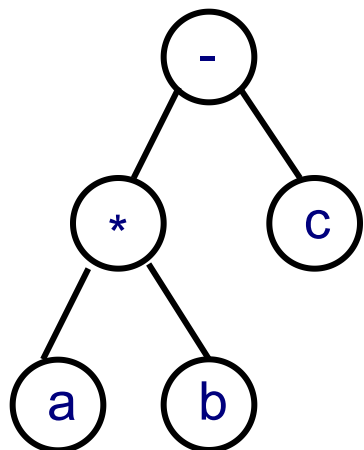
向下的箭头表示更深一层的递归调用



中序遍历二叉树

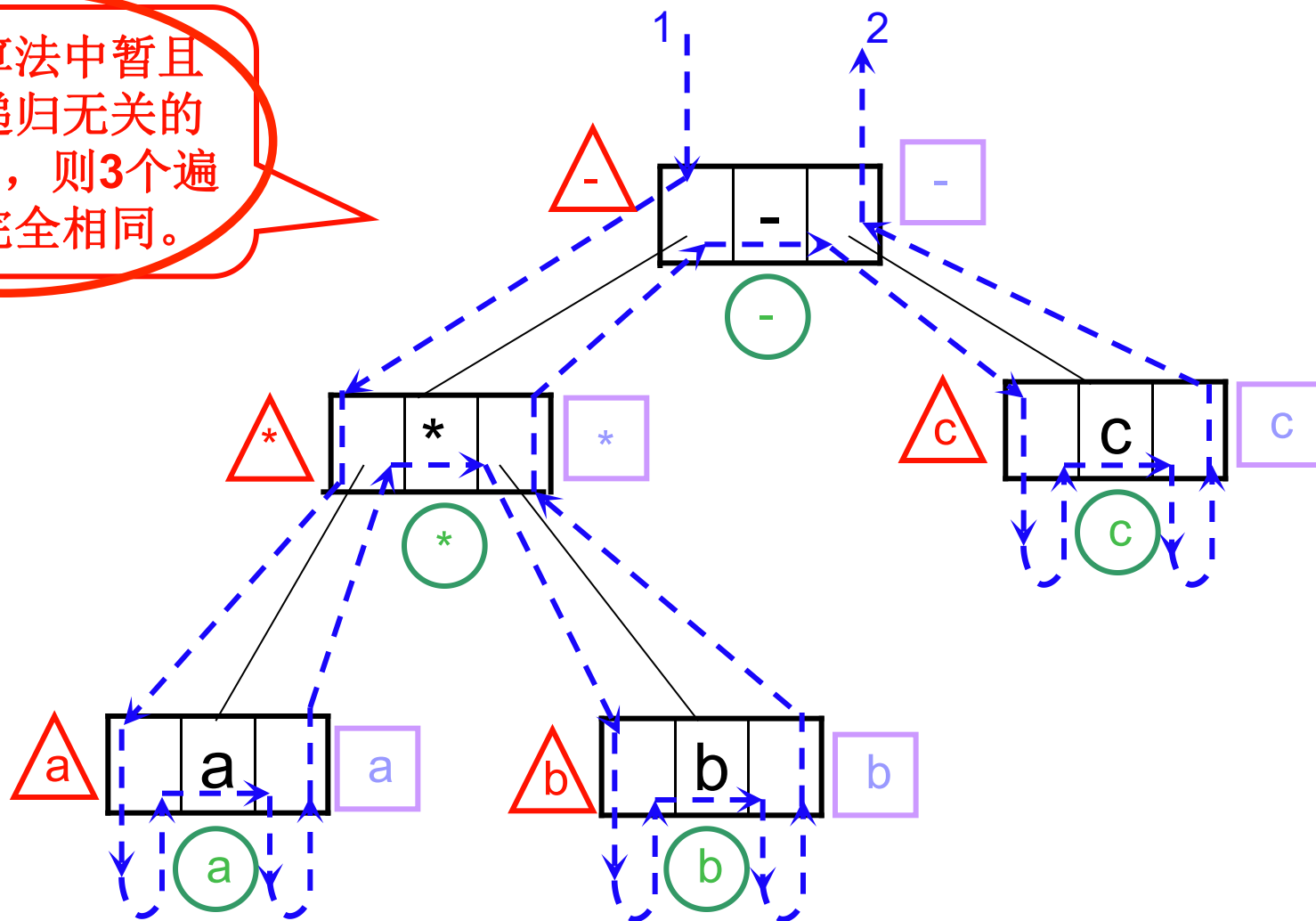


后序遍历二叉树



6.3.1 遍历二叉树

如果在算法中暂且抹去和递归无关的 **Visit** 语句，则3个遍历算法完全相同。



6.3.1 遍历二叉树

- 观察：中序遍历算法递归工作栈的状态变化。
 - (1) 当栈顶记录中的指针非空时，应遍历左子树，即指向左子树根的指针进栈。
 - (2) 若栈顶记录中的指针值为空，则应退至上一层。
 - 若是从左子树返回，则应访问当前层即栈顶指针上指针所指的根结点；
 - 若是从右子树返回，则表明当前层的遍历结束，应继续退栈。
- 由此可得以下两个中序遍历二叉树的非递归算法，先序遍历二叉树的非递归算法与此类似，但后序遍历二叉树的非递归算法要相对复杂一点。

中序遍历二叉树的非递归算法1:

```
Status InOrderTraverse(BiTree T, Status (*Visit)
    (TElemType e)) {
    InitStack(S); Push(S, T); //根指针进栈
    while (!StackEmpty(S)) {
        while (GetTop(S, p) && p) Push(S, p->lchild);
            //向左走到尽头

        Pop(S, p); //空指针退栈
        if (!StackEmpty(S)) { //访问结点, 向右一步
            Pop(S, p);
            if (!Visit(p->data)) return ERROR;
            Push(S, p->rchild);
        } //if
    } //while
    return OK;
}
```

中序遍历二叉树的非递归算法2:

```
Status InOrderTraverse (BiTree T, Status (*Visit)
    (TElemType e)) {
    InitStack (S); p=T;
    while (p||!StackEmpty (S)) {
        if (p) { //根指针进栈, 遍历左子树
            Push (S, p);
            p=p→lchild;
        }
        else { //根指针退栈, 访问根结点, 遍历右子树
            Pop (S, p);
            if (!Visit (p→data)) return ERROR;
            p=p→rchild;
        }
    }
    return OK;
}
```

遍历右子树时不再需要保存当前层的根结点。

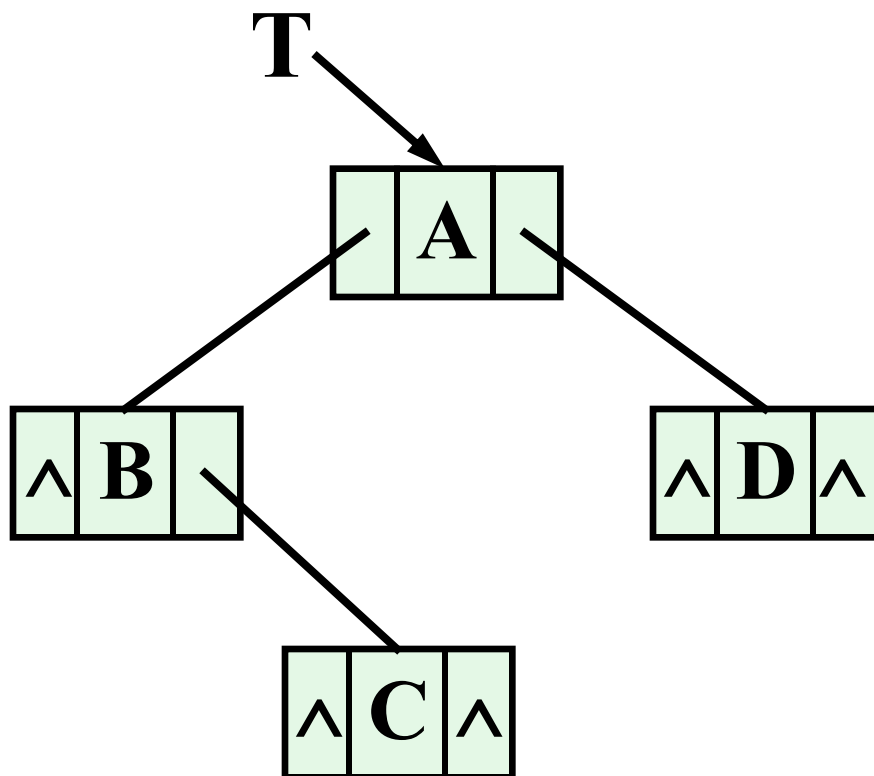
6.3.1 遍历二叉树

- “遍历”是二叉树各种操作的基础，可以在遍历过程中对结点进行各种操作，如按先序序列建立二叉树的二叉链表算法：

```
Status CreateBiTree(BiTree &T) {  
    scanf (&ch);  
    if (ch==' ') T=NULL;    //空格字符表示空树  
    else {  
        if (! (T=(BiTNode *) malloc (sizeof (BiTNode))))  
            exit (OVERFLOW);  
        T->data=ch;                //生成根结点  
        CreateBiTree (T->lchild);    //构造左子树  
        CreateBiTree (T->rchild);    //构造右子树  
    }  
    return OK;  
}
```

上页算法执行过程举例如下：

A **B** ■ **C** ■ ■ **D** ■ ■



6.3.1 遍历二叉树

遍历二叉树的时间复杂度和空间复杂度：

- 遍历二叉树的基本操作是访问结点，则不论按哪一种次序进行遍历，对含 n 个结点的二叉树，其时间复杂度均为 $O(n)$ 。
- 所需辅助空间为遍历过程中栈的最大容量，即树的深度，最坏情况为 n ，则空间复杂度也为 $O(n)$ 。

例子：由二叉树的先序和中序序列建树

- 仅知二叉树的先序序列“**abcdefg**” 能不能唯一确定一棵二叉树？
- 如果同时已知二叉树的中序序列“**cbdaegf**”，则会如何？

二叉树的先序序列

根

左子树

右子树

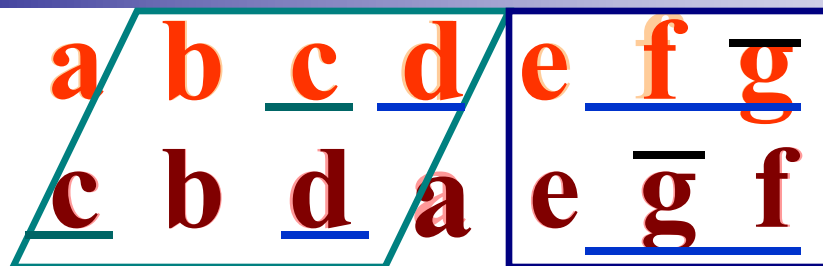
二叉树的中序序列

左子树

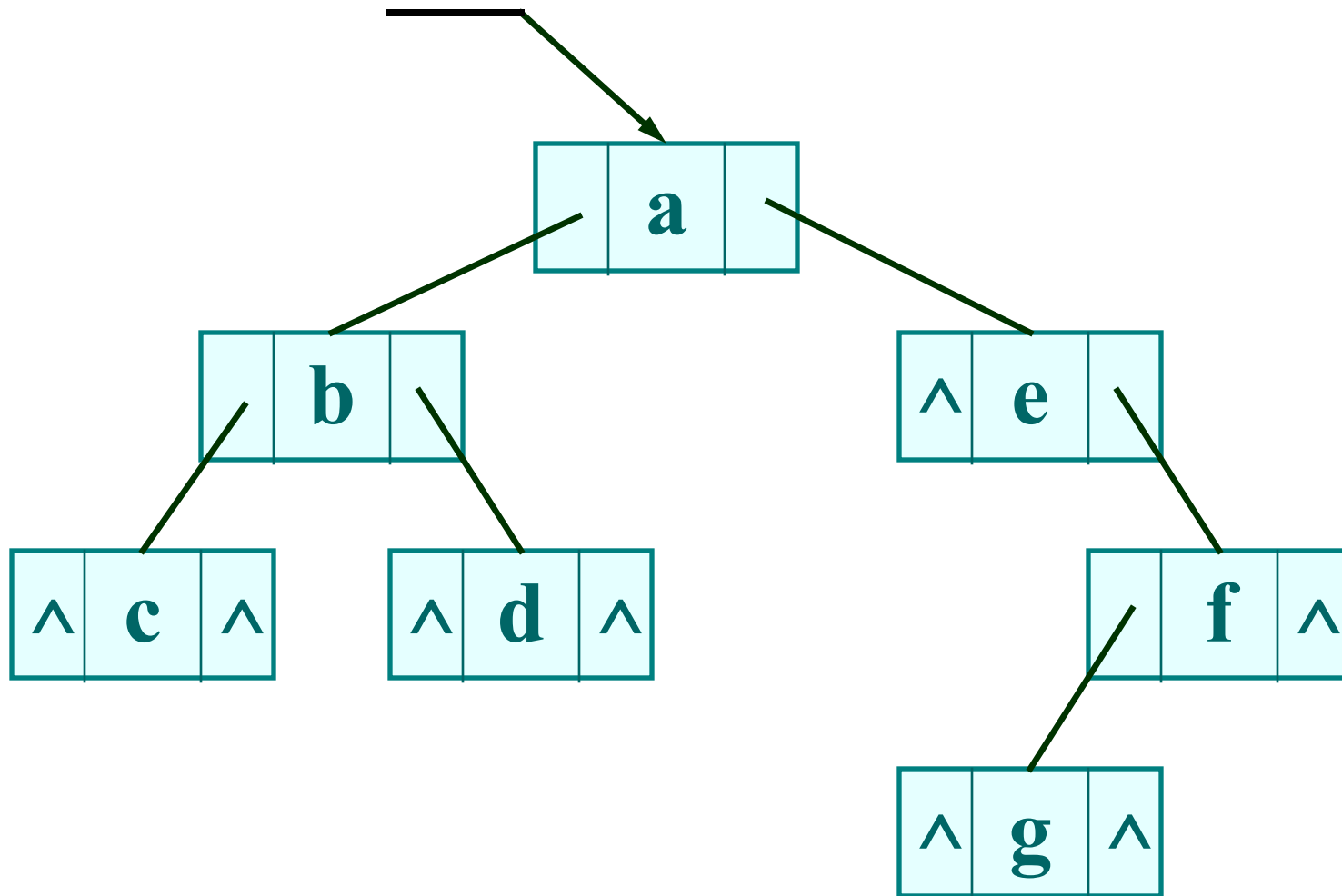
根

右子树

例如:



先序序列
中序序列




```

void CrtBT(BiTree& T, char pre[], char ino[], int ps, int is, int n ) {
// 已知pre[ps..ps+n-1]为二叉树的先序序列， ins[is..is+n-1]为二叉树的
//中序序列， 本算法由此两个序列构造二叉链表。
if (n==0) T=NULL;
else {
    k=Search(ino, pre[ps]); // 在中序序列中查询
    if (k== -1) T=NULL;
    else {
        T=(BiTNode*)malloc(sizeof(BiTNode));
        T->data = pre[ps];
        if (k==is) T->Lchild = NULL;
        else CrtBT(T->Lchild, pre[], ino[], ps+1, is, k-is );
        if (k==is+n-1) T->Rchild = NULL;
        else CrtBT(T->Rchild, pre[], ino[], ps+1+(k-is), k+1, n-(k-is)-1 );
    }
} //
} // CrtBT

```

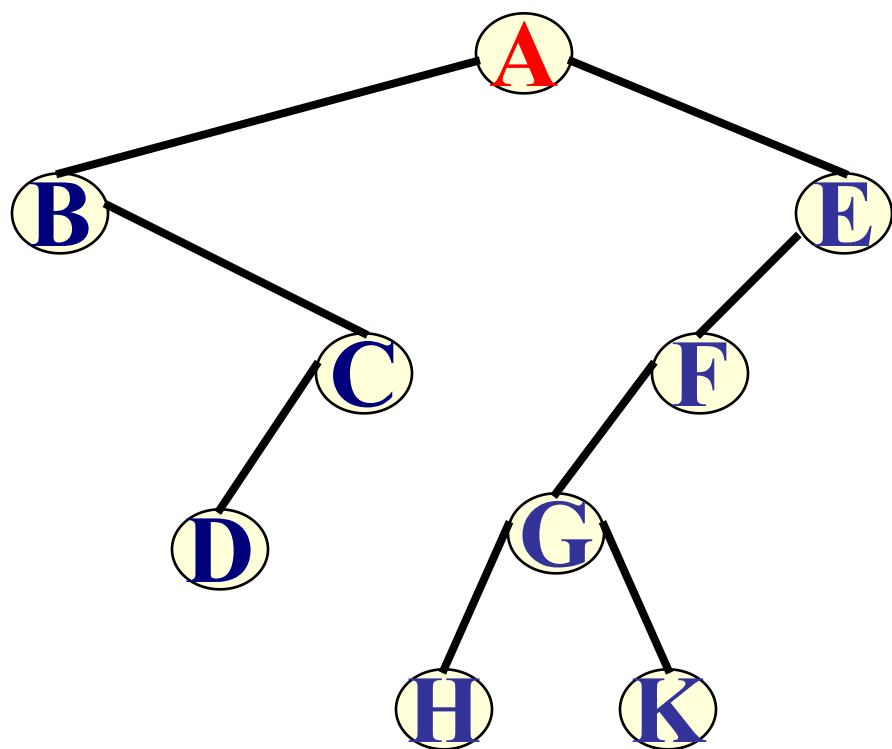
6.3.1 遍历二叉树

- 二叉树的先序序列和中序序列能够唯一确定一棵二叉树。
- 二叉树的后序序列和中序序列能够唯一确定一棵二叉树。
- 二叉树的先序序列和后序序列不能唯一确定一棵二叉树。

6.3.2 线索二叉树

问题：何谓线索二叉树？

遍历二叉树的结果是，求得结点的一个线性序列。



例如：

先序序列：

A B C D E F G H K

中序序列：

B D C A H G K F E

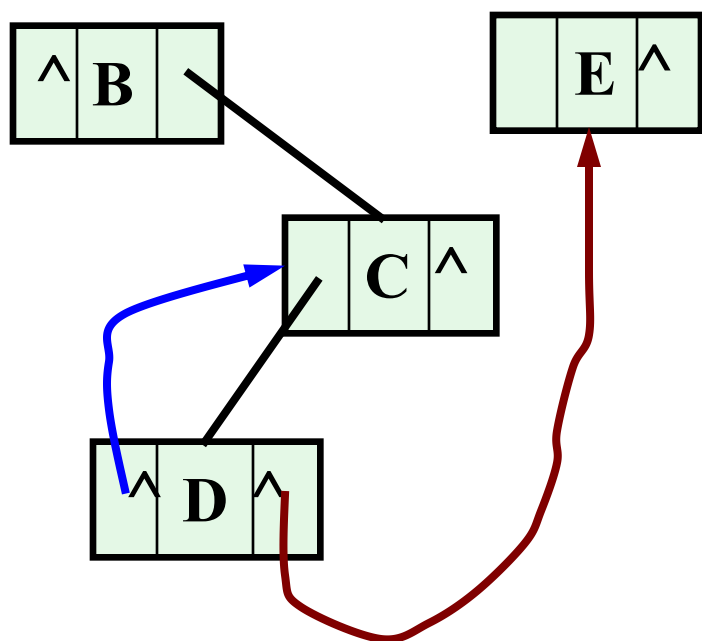
后序序列：

D C B H K G F E A

- 指向该线性序列中的“前驱”和“后继”的指针，称作“**线索**”

A B C D E F G H K

← (blue arrow from D to C) → (red arrow from D to E)



- 包含“线索”的存储结构，称作“**线索链表**”
- 与其相应的二叉树，称作“**线索二叉树**”

6.3.2 线索二叉树

- 当以二叉链表作为存储结构时, 只能找到结点的左右孩子的信息, 而不能得到结点的任一序列的前驱与后继信息, 这种信息只有在遍历的动态过程中才能得到。
- 为了能保存所需的信息, 在二叉链表的结点中增加两个标志域, 并作如下规定:
 - 若该结点的左子树不空, 则Lchild域的指针指向其左子树, 且左标志域的值为“指针 Link”; 否则, Lchild域的指针指向其“前驱”, 且左标志的值为“**线索 Thread**”。
 - 该结点的右子树不空, 则rchild域的指针指向其右子树, 且右标志域的值为“指针 Link”; 否则, rchild域的指针指向其“后继”, 且右标志的值为“**线索 Thread**”。

6.3.2 线索二叉树

lchild	LTag	data	RTag	rchild
--------	------	------	------	--------

LT ag = {
0 lchild 域指示结点的左孩子
1 lchild 域指示结点的前驱
0 rchild 域指示结点的右孩子
1 rchild 域指示结点的后驱

- 以这种结构构成的二叉链表作为二叉树的存储结构, 叫做**线索链表**, 其中指向结点前驱与后继的指针叫做**线索**。加上线索的二叉树称之为**线索二叉树**。对二叉树以某种次序遍历使其变为线索二叉树的过程叫做**线索化**。

6.3.2 线索二叉树

■ 二叉树的二叉线索存储表示:

```
typedef enum{Link,Thread}PointerTag;
```

//Link==0:指针,Thread==1:线索

```
typedef struct BiThrNode
```

```
{
```

```
TElemType data;
```

```
struct BiThrNode *lchild,*rchild; //左右指针
```

```
PointerTag LTag, Rtag; //左右标志
```

```
}BiThrNode,*BiThrTree;
```

6.3.2 线索二叉树

- 模仿线性表的存储结构, 在二叉树的线索链表上也添加一个头结点, 令其lchild域的指针指向二叉树的根结点, 其rchild域的指针指向中序遍历时访问的最后一个结点;
- 同时, 令二叉树中序序列中的第一个结点lchild域的指针和最后一个结点rchild域的指针均指向头结点, 就像为二叉树建立了一个双向线索链表, 就好比人在一个圆圈上走路, 有可能存在好走的可能性。

6.3.2 线索二叉树

图6.11为中序线索二叉树及其存储结构：

- (1) **寻找结点后继**：若结点右标志为“1”，则右链表线索，指示其后继；否则遍历右子树时访问的第一个结点（即右子树最左下结点）为其后继。
- (2) **寻找结点前驱**：若结点左标志为“1”，则左链表线索，指示其前驱；否则遍历左子树时最后访问的结点（即左子树最右下结点）为其前驱。

在中序线索二叉树上遍历
二叉树，不需要设栈

例如：对中序线索化链表的遍历算法

※ 中序遍历的第一个结点 ？

左子树上处于“最左下”（没有左子树）的结点。

※ 在中序线索化链表中结点的后继 ？

若无右子树，则为后继线索所指结点；否则为对其右子树进行中序遍历时访问的第一个结点。

Status InorderTraverse_Thr(BiThrTree T, Status(*visit)(TElemType))

```
{ //T指向头结点，头结点的lchild左链指向根结点，  
  //中序遍历二叉线索树的非递归算法，对每个数据元素调用函数Visit  
  P=T->lchild; //指向根结点  
  while(p!=T){ //空树或遍历结束时，p==T  
    while(p->LTag == Link) p=p->lchild; //第一个结点  
    //访问其左子树为空的结点  
    if(!visit(p->data)) return ERROR;  
    while(p->RTag == Thread&& p->rchild!=T) {  
      p=p->rchild; Visit(p->data); //访问后继结点  
    }  
    p= p->rchild; // p进至其右子树根  
  }  
  return OK;  
} //InorderTraverse_Thr
```

在中序线索二叉
树上遍历二叉树，
不需要设栈

6.3.2 线索二叉树

- 在**后序线索树**中寻找后继较复杂些，可分3种情况：
 - (1) 结点x是二叉树的根，则其后继为空；
 - (2) 结点x是其双亲的右孩子，或是其双亲的左孩子且其双亲没有右子树，则其后继为双亲结点；
 - (3) 结点x是其双亲的左孩子，且其双亲有右子树，则其后继为双亲的右子树上按后序遍历列出的第一个结点。
- 如图6.12。可见在**后序线索化树**上找后继时需知道**结点双亲**，即需要带标志域的三叉链表作存储结构。

6.3.2 线索二叉树

那么，又如何进行二叉树的线索化呢？

- **线索化的实质**是将二叉链表中的空指针改为指向前驱或后继的线索，而前驱或后继的信息只有在遍历时才能得到，因此**线索化的过程即为在遍历的过程中修改空指针的过程**。
- 为了记下遍历过程中访问结点的先后关系，附设一个指针pre始终指向刚刚访问过的结点，若指针p指向当前访问的结点，则pre指向它的前驱。
- 由此可得中序线索化的算法6.6和算法6.7。

中序遍历建立中序线索化链表的算法6.7 :

void InThreading(BiThrTree p)

{ // 对以p为根的非空二叉树进行线索化

if (p)

 {

 InThreading(p ->lchild); //左子树线索化

 if(!p ->lchild) {p -> LTag =Thread; p->lchild=pre;}

 //前驱线索

 if(!pre ->rchild){pre ->RTag =Thread;pre->rchild=p;}

 //后继线索

 pre=p; //保持pre指向p的前驱

 InThreading(p ->rchild); //右子树线索化

 }

}

中序遍历建立中序线索化链表的算法 6.6:

```
Status InorderThreading(BiThrTree &Thrt, BiThrTree T)
{ // 构建中序线索链表
    if(!(Thrt =(BiThrTree)malloc(sizeof(BiThrNode))))
        exit(OVERFLOW);
    Thrt ->LTag =Link; Thrt ->RTag =Thread; //建头结点
    Thrt ->rchild=Thrt;                      //右指针回指
    if (!T) Thrt ->lchild=Thrt; //若二叉树为空，则左指针回指
    else{
        Thrt ->lchild=T; pre=Thrt;
        InThrTreading(T); //中序遍历进行中序线索化
        pre ->rchild=Thrt; pre -> RTag =Thread; //最后一个结点线索化
        Thrt ->rchild=pre;
    }
    return OK;
} //InorderThreading
```

第六章 树和二叉树

6.1 树的定义和基本概念

6.2 二叉树

6.2.1 树的定义和基本术语

6.2.2 二叉树的性质

6.2.3 二叉树的存储结构

6.3 遍历二叉树

6.3.1 遍历二叉树

6.3.2 线索二叉树

6.4 树和森林

6.4.1 树的存储结构

6.4.2 森林与二叉树的转换

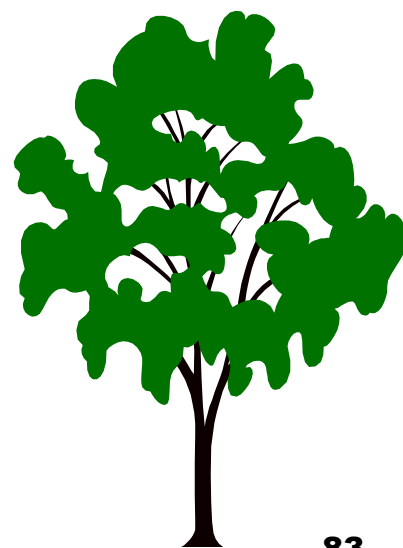
6.4.3 树和森林的遍历

6.6 赫夫曼树及其应用

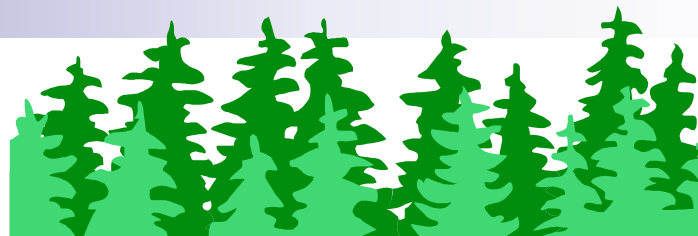
6.6.1 最优二叉树（赫夫曼树）

6.6.2 赫夫曼编码

6.7 回溯法与树的遍历



6.4 树和森林



6.4.1 树的存储结构

树的三种存储结构:

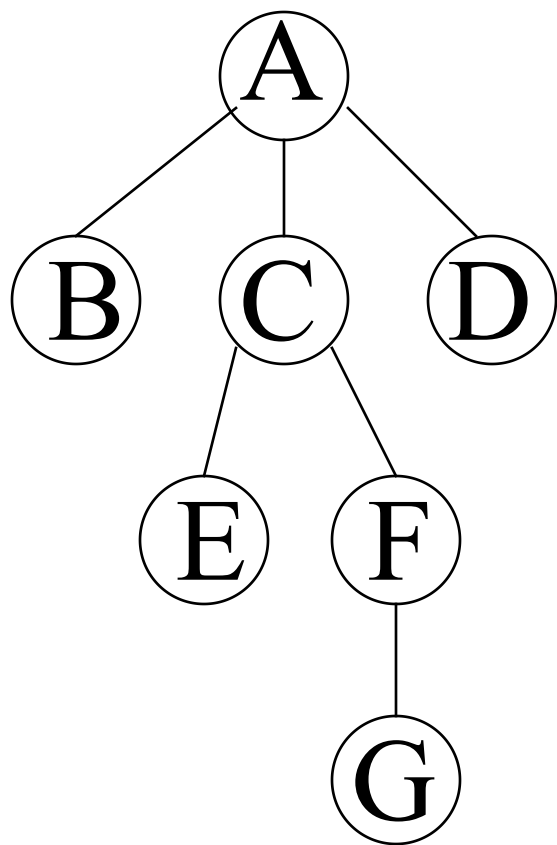
一、双亲表示法

二、孩子链表表示法

三、树的二叉链表(孩子-兄弟)

存储表示法

6.4.1 树的存储结构——双亲表示法



data parent

0	A	-1
1	B	0
2	C	0
3	D	0
4	E	2
5	F	2
6	G	5

$r=0$
 $n=6$

存储数组元素下标

6.4.1 树的存储结构——双亲表示法

- 以一组连续空间存储树的结点，并在每个结点中附设一个指示器指示其双亲结点在链表中的位置：

```
#define MAX_TREE_SIZE 100
```

```
typedef struct PTNode{
```

```
    TElemType data;
```

```
    int parent;    //双亲位置域
```

```
}PTNode;
```

```
typedef struct{
```

```
    PTNode nodes[MAX_TREE_SIZE];
```

```
    int r,n; //根的位置和结点数
```

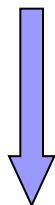
```
}PTree;
```

结点结构:

data	parent
------	--------

6.4.1 树的存储结构——双亲表示法

- 利用这种结构可以很方便地求出结点的双亲及其根，但是，在这种表示法中，求结点的孩子时需要遍历整个结构。



孩子表示法

6.4.1 树的存储结构——孩子表示法

- 由于孩子每个结点可能有多棵子树，则可用**多重链表**，即每个结点有多个指针域，其中每个指针指向一棵子树的根结点，此时链表中的结点可以有如下两种结点格式：

data	child1	child2		childd
------	--------	--------	-------	--------

data	degree	child1	child2		childd
------	--------	--------	--------	-------	--------

- 若采用第一种结点格式，则多重链表中的结点是**同构的**，其中 d 为树的度。由于树中很多结点的度小于 d ，所以链表中有很多空链域，空间较浪费。
- 若采用第二种结构，则多重链表中的结点是**不同构的**。其中**degree**为每个结点的度。然而，这也造成了操作的不方便。

6.4.1 树的存储结构——孩子表示法

- 另一种办法是把每个结点的孩子结点排列起来，看成一个线性表，且以单链表作存储结构，则 n 个结点有 n 个孩子链表。 n 个头指针又组成一个线性表，为了便于查找，可采用顺序存储结构。

C语言的类型描述:

孩子结点结构:

child	next
-------	------

```
typedef struct CTNode {  
    int      child;  
    struct CTNode *next;  
} *ChildPtr;
```

双亲结点结构

data	firstchild
------	------------

```
typedef struct {  
    Elem    data;  
    ChildPtr firstchild;  
            // 孩子链的头指针  
} CTBox;
```

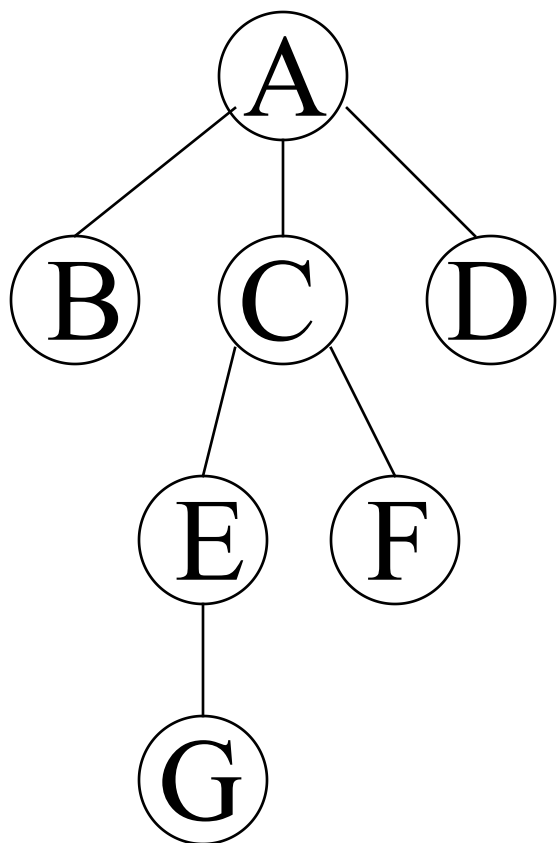

树结构:

```
typedef struct {  
    CTBox  nodes[MAX_TREE_SIZE];  
    int    n, r;  
    // 结点数和根结点的位置  
} CTree;
```

6.4.1 树的存储结构——孩子表示法

- 图6.14(a) 是图6.13的树的孩子链表。与双亲表示法相反，孩子表示法易于求孩子结点，而难以求双亲结点。我们可以把双亲表示法和孩子表示法结合起来，形成带双亲的孩子链表，如图6.14(b)。

6.4.1 树的存储结构——孩子表示法



data firstchild

0	A	-1		→	1	→	2	→	3	Λ
1	B	0	Λ							
2	C	0		→	4	→	5	Λ		
3	D	0	Λ							
4	E	2		→	6	Λ				
5	F	2	Λ							
6	G	4	Λ							

r=0
n=6

6.4.1 树的存储结构——孩子兄弟表示法

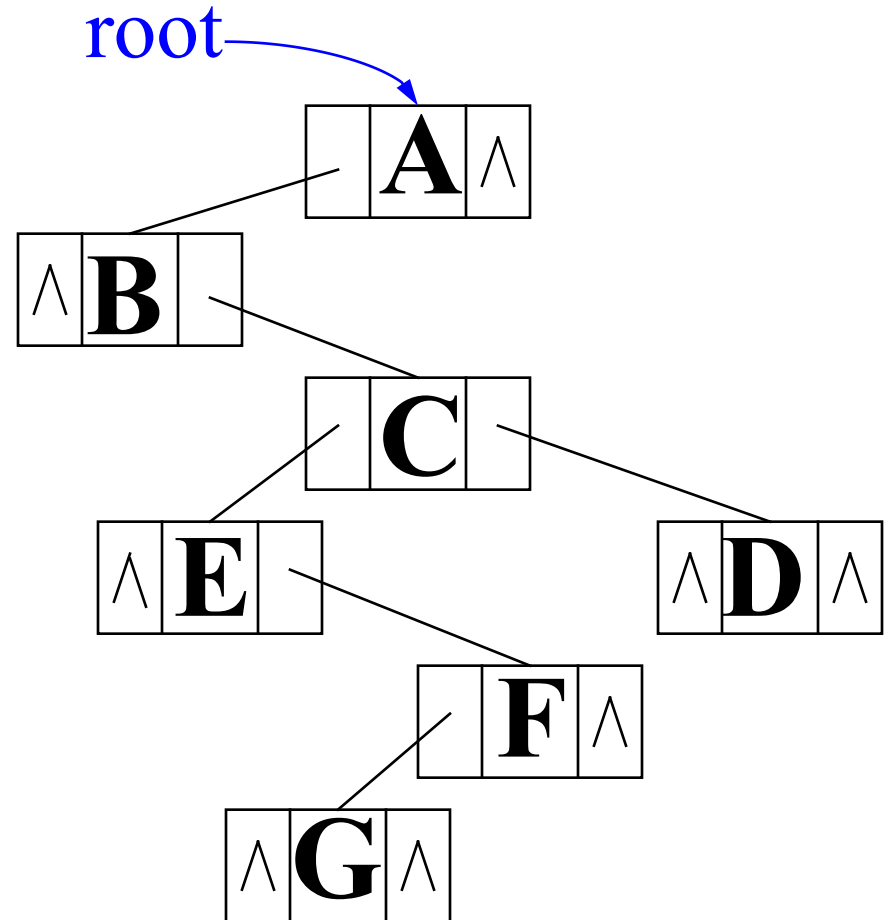
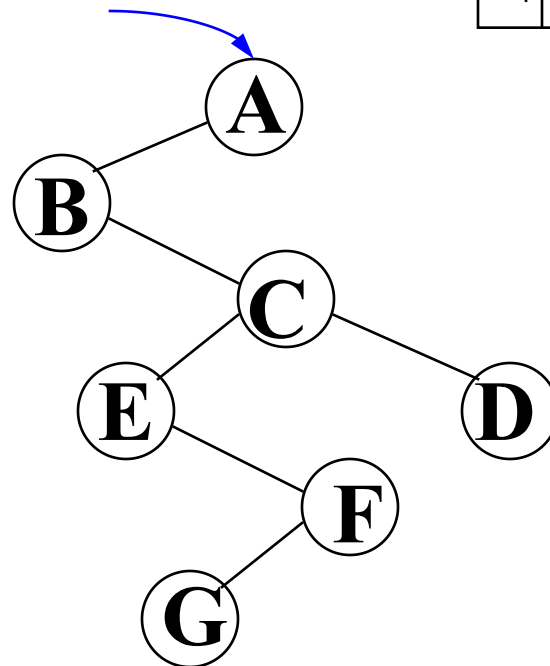
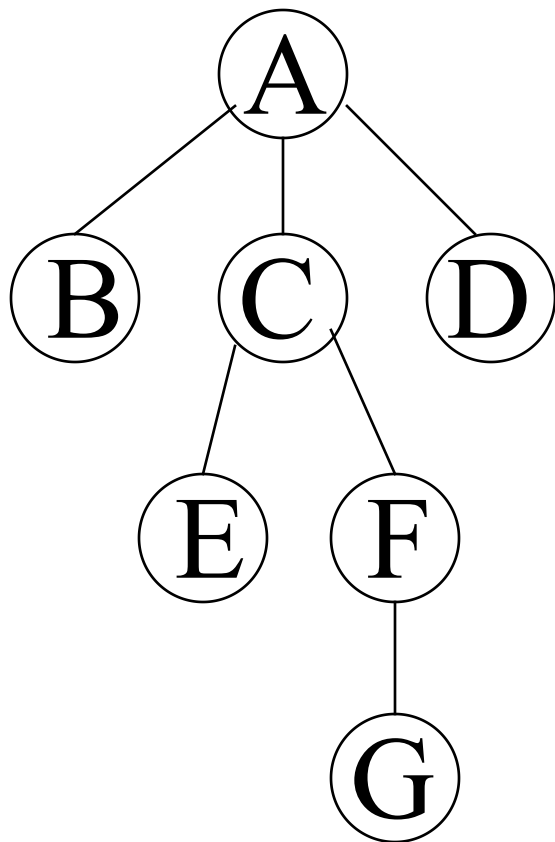
- 又称二叉树表示法，或二叉链表表示法，即以二叉树作树的存储结构。链表中结点的两个链域分别指向该结点的第一个孩子和下一个兄弟结点。

结点结构:

firstchild	data	nextsibling
------------	------	-------------

```
typedef struct CSNode{  
    ElemType data;  
    struct CSNode *firstchild, *nextsibling;  
}CSNode, *CSTree;
```

例子:



6.4.1 树的存储结构——孩子兄弟表示法

- 利用这种结构，易于找到结点的孩子。
 - 例如：要访问结点x的第i个孩子，则只要先从firstchild域找到第1个孩子结点，然后沿着孩子结点的nextsibling域连续走i-1步，便可找到结点x的第i个孩子。
- 如果每个结点增设一个PARENT域，则同样能方便地实现PARENT(T,x)操作。
- 观察上页的图，我们可以发现，树也可以用二叉树的存储方式来存储。这意味着，树与二叉树之间存在某种关联关系。

6.4.2 森林与二叉树的转换

- 由于二叉树和树都可用二叉链表作为存储结构，它们两者的物理结构相同，只是解释不同而已。图6.16直观地展示出树与二叉树的对应关系。
- 由树的二叉链表定义可知，任何一棵和树对应的二叉树，其右子树必空（因为根无兄弟结点）。
- 若把森林中第二棵树的根结点看成是第一棵树的根结点的兄弟，则同样可导出森林和二叉树的对应关系。如图6.17。
- 这个一一对应的关系使得森林与二叉树可以相互转换，其形式定义如下：

6.4.2 森林与二叉树的转换

■ 森林转换成二叉树:

如果 $F=\{T_1, T_2, \dots, T_m\}$ 是森林，则可按如下规则转换成一棵二叉树 $B=\{\text{root}, LB, RB\}$ 。

(1) 若 F 为空，即 $m=0$ ，则 B 为空树。

(2) 若 F 非空，即 $m \neq 0$ ，则

- B 的根 root 即为森林中第一棵树的根 $\text{ROOT}(T_1)$ ；
- B 的左子树 LB 是从 T_1 中根结点的子树森林 $F_1=\{T_{11}, T_{12}, \dots, T_{1m1}\}$ 转换而成的二叉树；
- B 的右子树 RB 是从森林 $F'=\{T_2, \dots, T_m\}$ 转换而成的二叉树。

6.4.2 森林与二叉树的转换

■ 二叉树转换成森林:

如果 $\mathbf{B}=\{\text{root}, \mathbf{LB}, \mathbf{RB}\}$ 是一棵二叉树, 则可按如下规则转换成森林 $\mathbf{F}=\{\mathbf{T}_1, \mathbf{T}_2, \dots, \mathbf{T}_m\}$:

(1) 若 $\mathbf{B}$ 为空, 即 $\mathbf{F}$ 为空。

(2) 若 $\mathbf{B}$ 非空, 则

- $\mathbf{F}$ 中第一棵树 $\mathbf{T}_1$ 的根 $\mathbf{ROOT}(\mathbf{T}_1)$ 即为二叉树 $\mathbf{B}$ 的根 root ;
- $\mathbf{T}_1$ 中根结点的子树森林 $\mathbf{F}_1$ 是由 $\mathbf{B}$ 的左子树 $\mathbf{LB}$ 转换而成的森林;
- $\mathbf{F}$ 中除 $\mathbf{T}_1$ 之外其余树组成的森林 $\mathbf{F}'=\{\mathbf{T}_2, \dots, \mathbf{T}_m\}$ 是由 $\mathbf{B}$ 的右子树 $\mathbf{RB}$ 转换而成的森林。

6.4.2 森林与二叉树的转换

- 由此，树的各种操作均可对应二叉树的操作来完成。
- 应当注意的是：和树对应的二叉树，其左、右子树的概念已改变为：左是孩子，右是兄弟。

6.4.3 树和森林的遍历

树的遍历可有三条搜索路径:

- 先根(次序)遍历:

若树不空, 则先访问根结点, 然后依次先根遍历各棵子树。

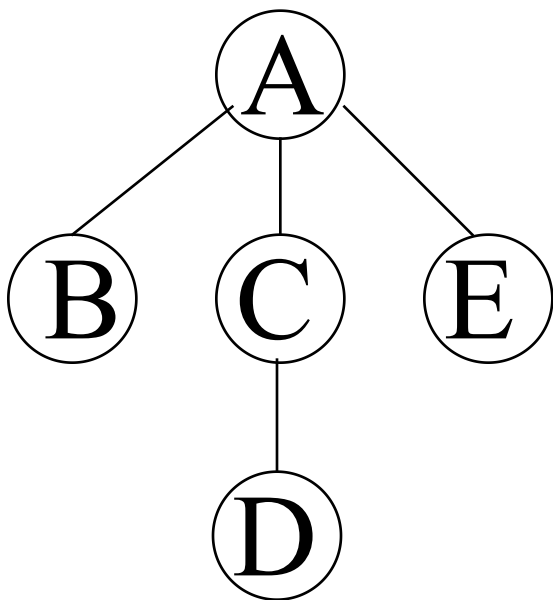
- 后根(次序)遍历:

若树不空, 则先依次后根遍历各棵子树, 然后访问根结点。

- 按层次遍历:

若树不空, 则 自上而下 自左至右 访问树中每个结点。

6.4.3 树和森林的遍历



■ 树的遍历例子:

□ 先根（次序）遍历:

A B C D E

□ 后根（次序）遍历:

B D C E A

□ 按层次遍历:

A B C E D

■ 例子:

- 先根遍历时顶点的访问次序:

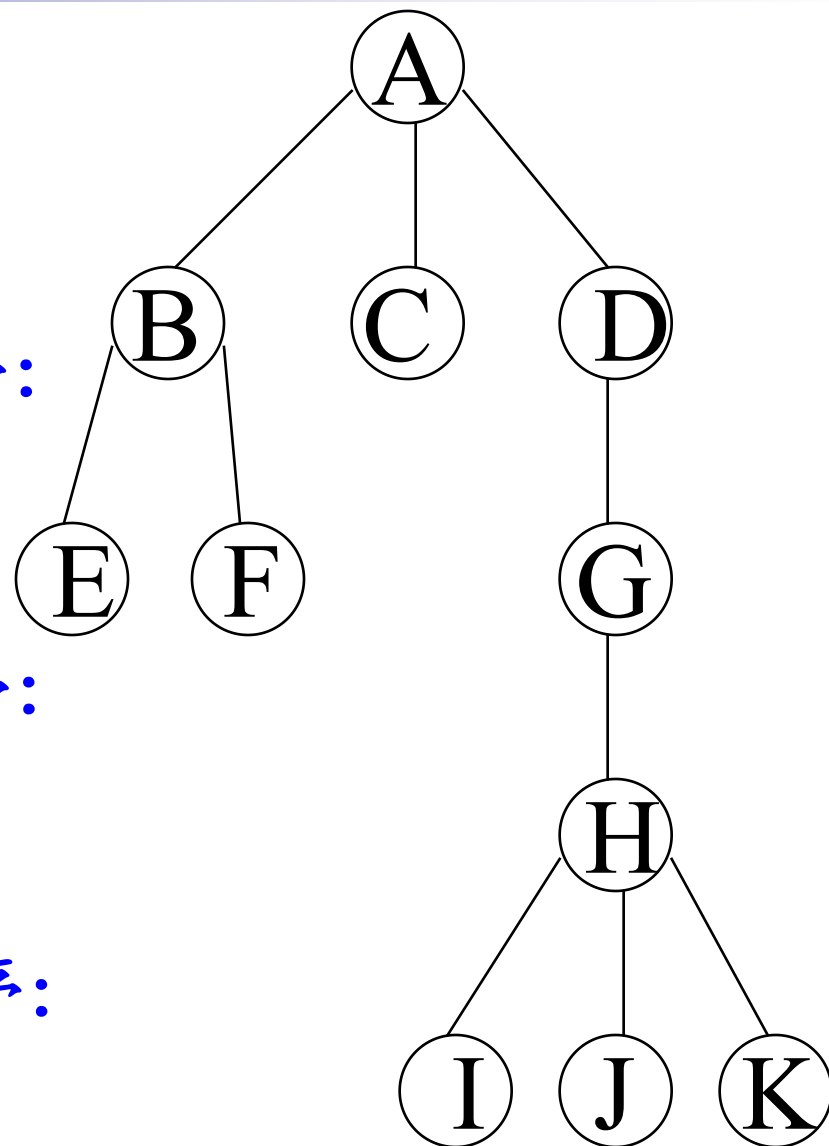
A B E F C D G H I J K

- 后根遍历时顶点的访问次序:

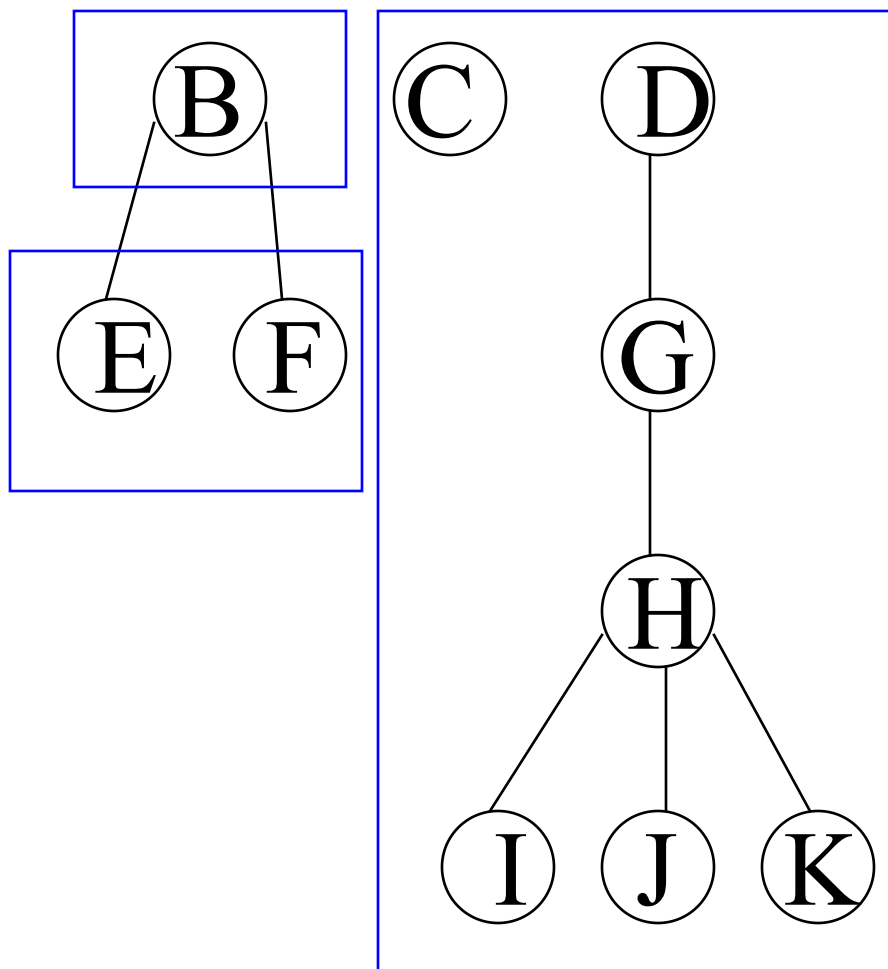
E F B C I J K H G D A

- 层次遍历时顶点的访问次序:

A B C D E F G H I J K



6.4.3 树和森林的遍历



森林由三部分构成：

1. 森林中第一棵树的根结点；
2. 森林中第一棵树的子树森林；
3. 森林中其它树构成的森林。

6.4.3 树和森林的遍历

- 由森林和树相互递归的定义，我们可以推出森林的两种遍历方法：

1、先序遍历森林：

- (1) 访问森林中第一棵树的根结点；
- (2) 先序遍历第一棵树中根结点的子树森林；
- (3) 先序遍历除去第一棵树之后剩余的树构成的森林

对于图6.17，先序序列为：**A B C D E F G H I J**

- 即：依次从左至右对森林中的每一棵树进行先根遍历。

6.4.3 树和森林的遍历

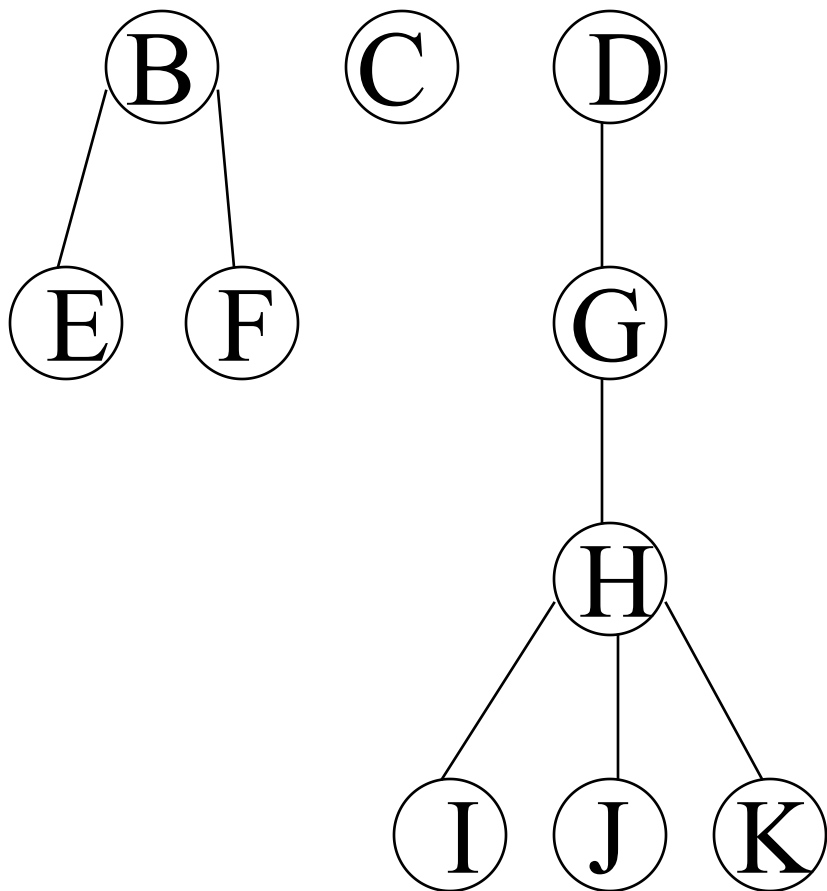
2、中序遍历森林：

- (1) 中序遍历第一棵树中根结点的子树森林；
- (2) 访问森林中第一棵树的根结点；
- (3) 中序遍历除去第一棵树之后剩余的树构成的森林。

图6.17的中序序列为：**B C D A F E H J I G**

- 即：依次从左至右对森林中的每一棵树进行后根遍历。

6.4.3 树和森林的遍历



■ 森林的遍历例子：

□ 先序遍历：

B E F C D G H I J K

□ 中序遍历：

E F B C I J K H G D

6.4.3 树和森林的遍历

■ 森林遍历与二叉树遍历的对应关系：

森林的先序和中序遍历分别对应二叉树的先序和中序遍历。

树	森林	二叉树
先根遍历	先序遍历	先序遍历
后根遍历	中序遍历	中序遍历

■ 由此可见，相当部分森林与树的操作，可以归结为二叉树的操作。

第六章 树和二叉树

6.1 树的定义和基本概念

6.2 二叉树

6.2.1 树的定义和基本术语

6.2.2 二叉树的性质

6.2.3 二叉树的存储结构

6.3 遍历二叉树

6.3.1 遍历二叉树

6.3.2 线索二叉树

6.4 树和森林

6.4.1 树的存储结构

6.4.2 森林与二叉树的转换

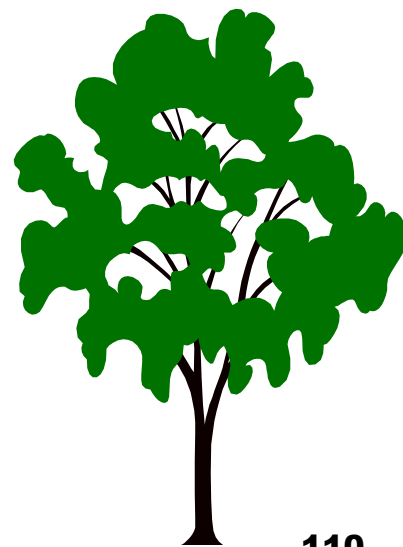
6.4.3 树和森林的遍历

6.6 赫夫曼树及其应用

6.6.1 最优二叉树（赫夫曼树）

6.6.2 赫夫曼编码

6.7 回溯法与树的遍历



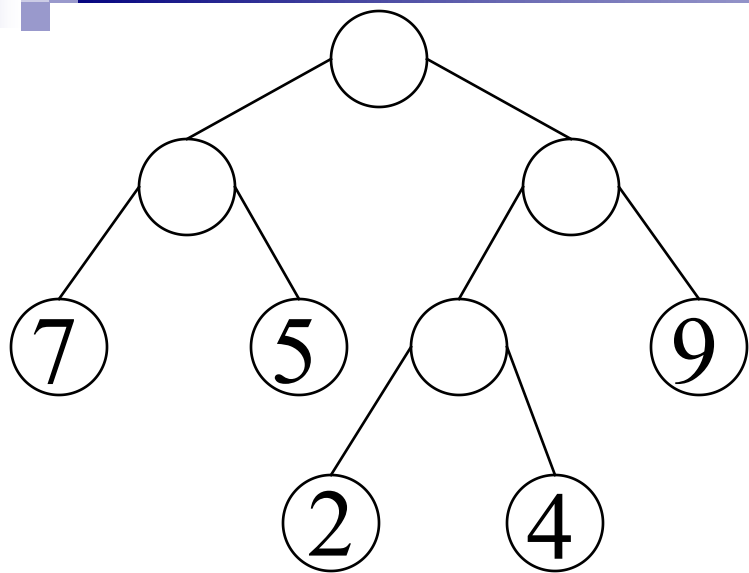
6.6 赫夫曼树及其应用

- 赫夫曼树(Huffman)，又称最优树，是一类带权路径长度最短的树，有着广泛的应用。

6.6.1 最优二叉树（赫夫曼树）

- 路径长度：从树中一个结点到另一个结点之间的分支构成这两个结点之间的路径，路径上的分支数目称为路径长度。
- 树的路径长度：从树根到每一结点的路径长度之和。
- 树的带权路径长度：考虑带权值的结点。树中所有叶子结点的带权路径长度之和，记作

$$WPL(\text{Weighted Path Length})=w_1l_1+w_2l_2+\dots+w_nl_n$$

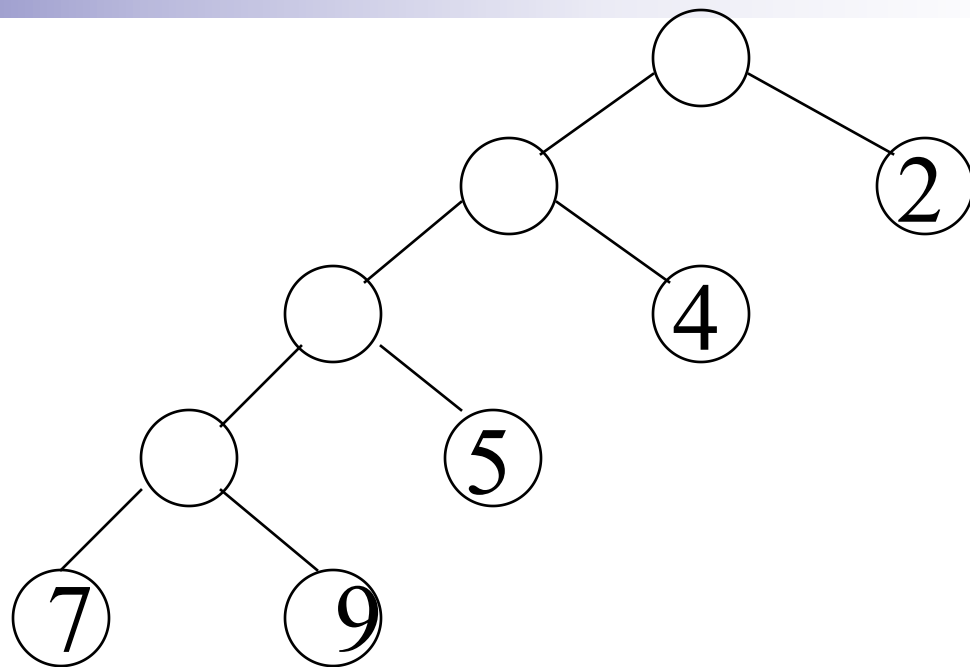


WPL(T)=

$7 \times 2 + 5 \times 2 + 2 \times 3 +$

$4 \times 3 + 9 \times 2$

$= 60$



WPL(T)=

$7 \times 4 + 9 \times 4 + 5 \times 3 +$

$4 \times 2 + 2 \times 1$

$= 89$

6.6.1 最优二叉树（赫夫曼树）

- 例如图6.22中的3棵二叉树，都有4个结点a, b, c, d，分别带权7、5、2、4，它们的带权路径长度分别为：

(a) $WPL=7*2+5*2+2*2+4*2=36$

(b) $WPL=7*3+5*3+2*1+4*2=46$

(c) $WPL=7*1+5*2+2*3+4*3=35$

- 其中以(c)树为最小。可以验证(c)树恰为赫夫曼树。

6.6.1 最优二叉树（赫夫曼树）

例子：构造一个将百分制转换成五级分制的程序。
该程序可以用条件语句来完成，如图6.23(a)图。

- 在实际生活中，学生成绩在5个等级上的是**不均匀的**。假设其分布规律如下：

分数：0—59	60—69	70—79	80—89	90—100
比例：0.05	0.15	0.40	0.30	0.10

以频数为权值

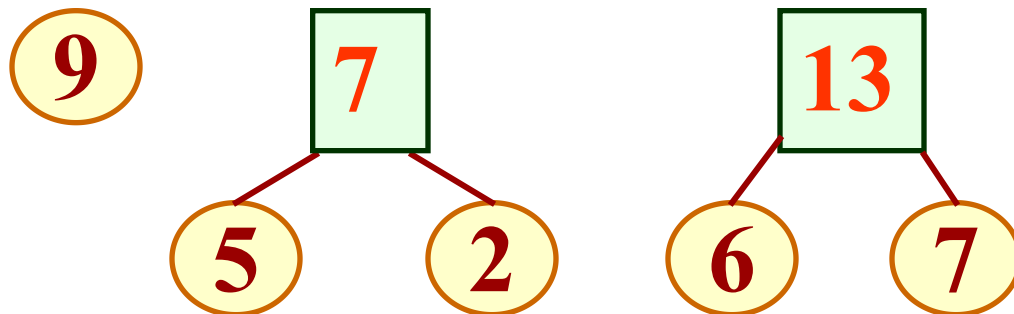
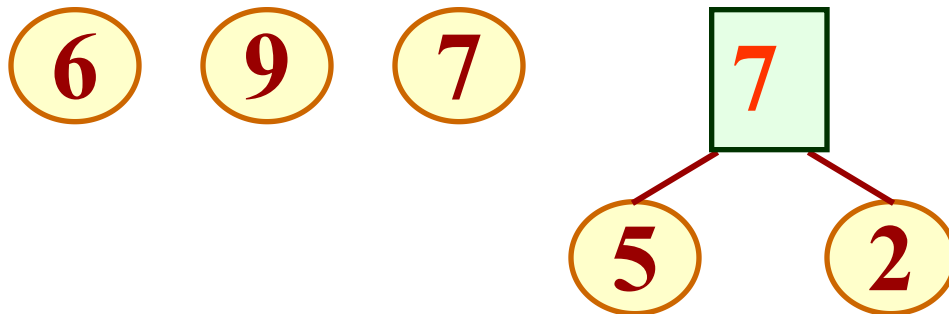
- 假定以5、15、40、30和10为权构造一棵有5个叶子结点的**赫夫曼树**，则可得到如图6.23(b)的判定树。它可使大部分的数据经过较少的比较次数得出结果。
- 图6.23(c)把图6.23(b)每个结点的两次比较化为每个结点进行一次比较，进一步演示了赫夫曼树的优越性。

6.6.1 最优二叉树（赫夫曼树）

■ 构造赫夫曼树的算法（以二叉树为例）：

- (1) 根据给定的 n 个权值 $\{w_1, w_2, \dots, w_n\}$ 构成 n 棵二叉树的集合 $F=\{T_1, T_2, \dots, T_n\}$ ，其中每棵二叉树 T_i 中只有一个带权为 w_i 的根结点，其左右子树为空。
 - (2) 在 F 中选取两棵根结点的权值最小的树作为左右子树构造一棵新的二叉树，且置新的二叉树的权值为其左、右子树上根结点的权值之和。
 - (3) 在 F 中删除这两棵树，同时将新得到的二叉树加入 F 中。
 - (4) 重复(2)和(3)，直到 F 只含一棵树为止。这棵树便为赫夫曼树。
- 图6.24展示了图6.22(c)的赫夫曼树的构造过程。

例如：已知权值 $W=\{ 5, 6, 2, 9, 7 \}$



9

7

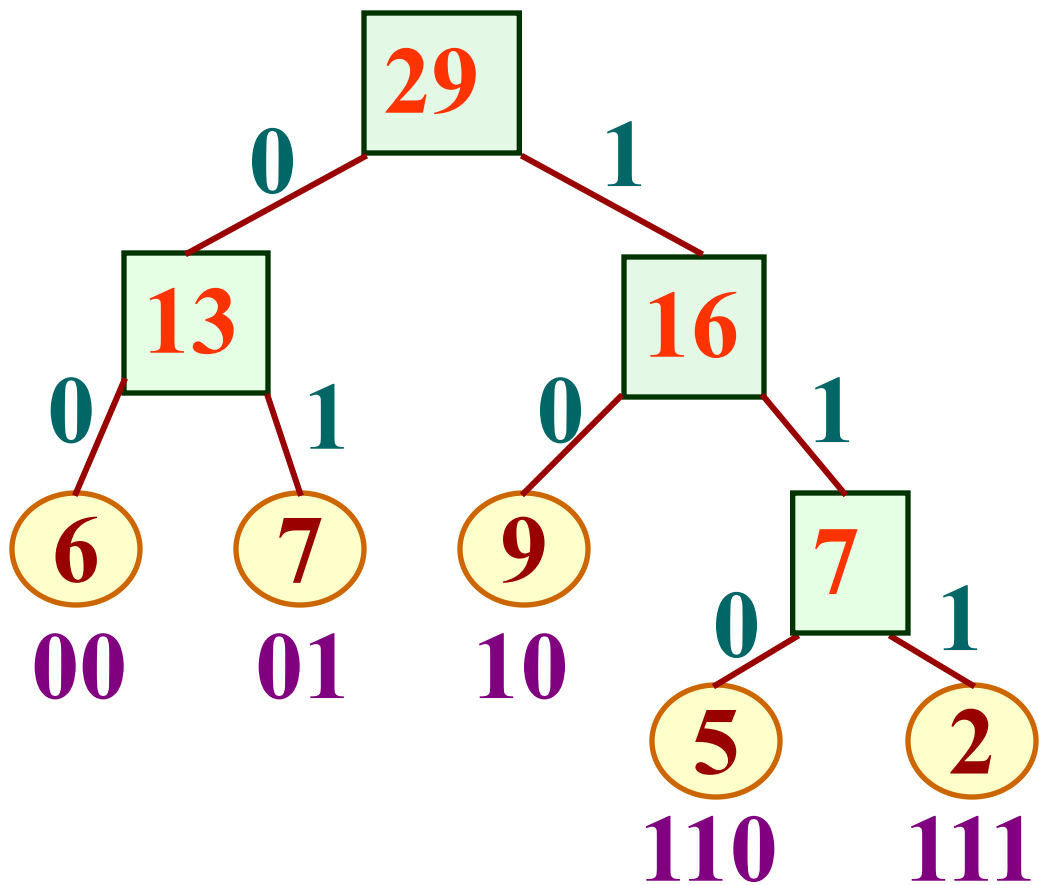
5

2

13

6

7



6.6.2 赫夫曼编码——赫夫曼树的应用之一

- 使用电报进行远程通信，需要将需传送的文字转换成二进制的字符组成的字符串。

□ 例如，需要传送的电文为' **A B A C C D A**'，总共只有4个字符，可以编码为：

A	B	C	D
00	01	10	11

□ 则上述电文便为' **00010010101100**'，总长14位，对方接收时，可以两位一组进行译码。

6.6.2 赫夫曼编码——赫夫曼树的应用之一

- 在传送电文时，希望总长尽可能地短，如果对每个字符设计长度不等的编码，且让电文中出现次数较多的字符采用尽可能短的编码，则传送电文的总长便可减少，如：

A	B	C	D
0	00	1	01

- 上述电文便可编码为总长为9：'000011010'。
 - 但这时候，又可能带来歧义。比如对于'0000'就有多种译法，可以是'AAAA'，也可以是'ABA'等。
- 因此，若要设计长短不一的编码，则必须是任一个字符的编码都不是另一个字符的编码的前缀，这种编码称为前缀编码。

6.6.2 赫夫曼编码——赫夫曼树的应用之一

■ 利用二叉树来设计二进制的前缀编码：

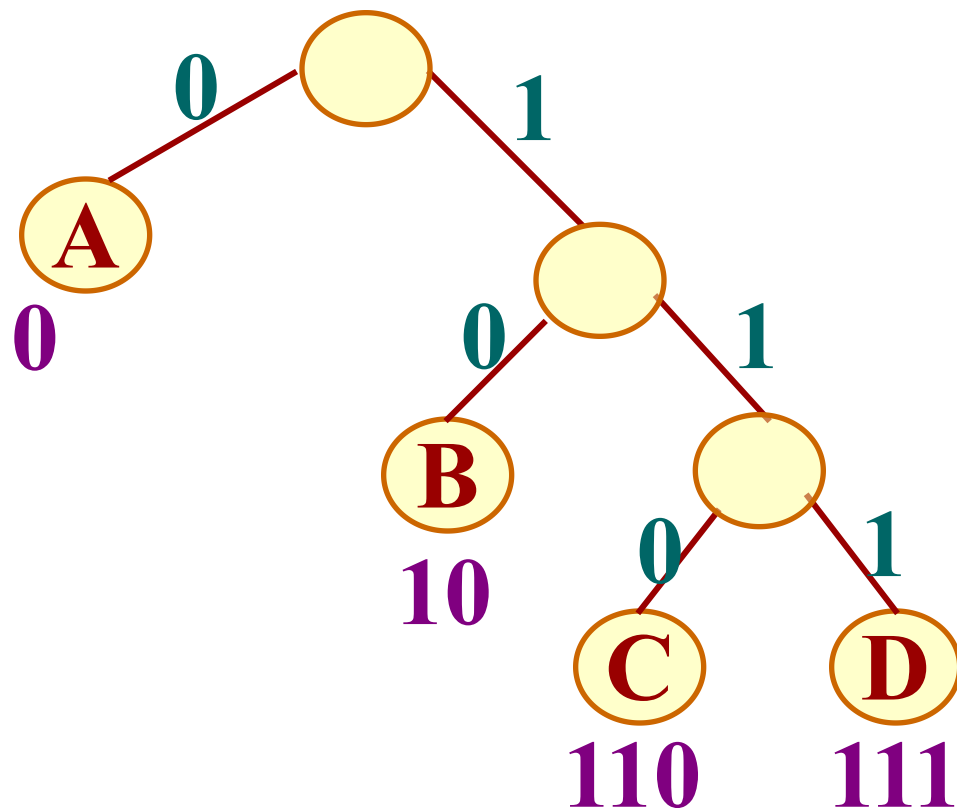
如图所示的二叉树，约定左分支表示字符‘0’，右分支表示字符‘1’，则可以从根结点到叶子结点的路径上分支字符组成的字符串作为该叶子结点的编码。

A(0)

B(10)

C(110)

D(111)



6.6.2 赫夫曼编码——赫夫曼树的应用之一

- 如何得到使电文总长最短的二进制编码呢？
- 假设每种字符在电文中出现的次数为 w_i ，其编码长度为 l_i ，电文中只有 n 种字符，则电文编码总长为 $w_1l_1+w_2l_2+\dots+w_nl_n$ 。
将电文编码总长建模为二叉树的带权路径长度
- 对应到二叉树上，若置 w_i 为叶子结点的权， l_i 为从根到叶子的路径长度。则 $w_1l_1+w_2l_2+\dots+w_nl_n$ 恰为二叉树上带权路径长度WPL。
- 由此可见，设计电文总长最短的二进制前缀编码即为以 n 种字符出现的频率作权，设计一棵赫夫曼树的问题，由此得到的二进制前缀编码便称为赫夫曼编码。
- 以下讨论具体做法。

6.6.2 赫夫曼编码——赫夫曼树的应用之一

■ 算法分析:

- 由于赫夫曼树中没有度为1的结点（这类树又称为正则二叉树），则一棵有n个叶子结点的赫夫曼树共有 $2n-1$ 个结点，可以存储在一个大小为 $2n-1$ 的一维数组中。
- 如何选定结点结构？
 - 由于在构成赫夫曼树之后，为求编码需从叶子结点出发走一条从叶子结点到根结点的路径；
 - 而为译码需从根出发走一条从根到叶子结点的路径。则对每个结点而言，需知道结点的双亲信息和孩子信息。

6.6.2 赫夫曼编码——赫夫曼树的应用之一

- 故可设定下述存储结构:

```
typedef struct
```

```
{
```

```
    unsigned int weight;
```

```
    unsigned int parent, lchild, rchild;
```

```
}HTNode , *HuffmanTree; //赫夫曼树
```

```
typedef char **HuffmanCode; //赫夫曼编码表
```


■ 赫夫曼编码的算法描述(见P147算法6.12):

输入: w 存放 n 个字符的权值

输出: 赫夫曼树 HT , 求赫夫曼编码 HC

[初始化]

总结点数为 $m=2*n-1$; 申请 m 个结点空间的赫夫曼树 HT ; 用 w 为前 n 个结点的权值赋值; 后面的结点权值初始为 0。这些结点的双亲和孩子皆为 0。

[建赫夫曼树 HT]

i 从 $n+1$ 到 m , 执行如下循环:

在 $HT[1..i-1]$ 中选择父亲信息为 0 且权值最小的两个结点 $s1$ 和 $s2$, 以 i 作为它们的双亲, 并修改他们的双亲信息为 i , 修改 i 的孩子信息为 $s1$ 和 $s2$, i 的权重为 $s1$ 和 $s2$ 的权重之和。

[求赫夫曼编码HC]

分配n个字符编码的头指针向量HC;

i从1到n, 执行如下循环:

沿着i的父亲信息上溯, 直到根:

在每一层, 如果为左孩子则记下 '0';

在每一层, 如果为右孩子则记下 '1';

把上溯过程得到的字符串逆向写入HC[i]。

[算法结束]

这样由叶子到根, 逆向处理, 可以得到赫夫曼编码HC。其中, HC的前n个分量表示叶子结点, 其余结点为非终端结点, 最后一个结点为根结点。

6.6.2 赫夫曼编码——赫夫曼树的应用之一

- 由根出发，遍历整棵赫夫曼树，求得各个叶子所表示的字符的赫夫曼编码，如算法6.13，其中借用结点的weight域来作结点状态标志(weight为0表示向左，1表示向右，2表示退回)：

[初始化]

分配n个字符编码的头指针向量HC；

HT中的m个结点信息的weight赋值为0。

[求赫夫曼编码HC]

p从m（根结点）到0(根结点的双亲)，执行如下循环：

如果HT[p].weight==0（向左），则HT[p].weight赋值为1

如果p有左孩子，则p=p的左孩子，并记录字符“0”。

否则如果p没有右孩子，则把沿途的字符串记入HC[p]。

如果HT[p].weight==1（向右），则HT[p].weight赋值2，p赋值为p的右孩子，并记录字符“1”。

如果HT[p].weight==2（退回），则HT[p].weight赋值为0，p赋值为p的双亲。

[算法结束]

第六章 树和二叉树

6.1 树的定义和基本概念

6.2 二叉树

6.2.1 树的定义和基本术语

6.2.2 二叉树的性质

6.2.3 二叉树的存储结构

6.3 遍历二叉树

6.3.1 遍历二叉树

6.3.2 线索二叉树

6.4 树和森林

6.4.1 树的存储结构

6.4.2 森林与二叉树的转换

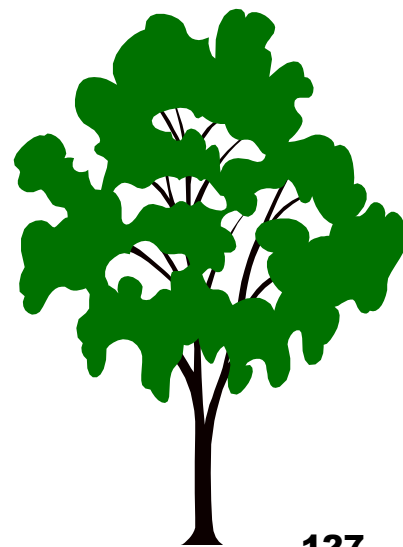
6.4.3 树和森林的遍历

6.6 赫夫曼树及其应用

6.6.1 最优二叉树（赫夫曼树）

6.6.2 赫夫曼编码

6.7 回溯法与树的遍历



6.7 回溯法与树的遍历

- 在程序设计中，有相当一类求一组解，或求全部解或求最优解的问题，例如八皇后、迷宫等，不是根据某种确定的计算法则，而是**利用试探和回溯（BackTracking）的搜索技术求解**。
- **回溯法**是设计递归过程的一种重要方法，它的求解过程实质上是一个**先序遍历**一棵“状态树”的过程，只是这棵树并不是遍历前预先建立，而是隐含在遍历过程中的。如果认识到这点，很多问题的递归过程也就迎刃而解。

6.7 回溯法与树的遍历

- 例6—3求含n个元素的集合的幂集。

如 $A=\{1,2,3\}$ ，则 $P(A)=\{\Phi, \{1\}, \{2\}, \{3\}, \{1,2\}, \{1,3\}, \{2,3\}, \{1,2,3\}\}$

- 集合的每个元素只有两种状态：
 - 或者属幂集的元素，
 - 或不属于幂集的元素。
- 求幂集 $P(A)$ 的元素的过程：可看成是依次对 A 中元素进行“取”或“舍”的过程，并且可以用一棵如图6.28所示的二叉树来表示。
 - 其中根结点表示幂集元素的初始状态（为空）；叶子结点表示它的终结状态（对应于幂集的8个元素）。

6.7 回溯法与树的遍历

- 因此，求幂集元素的过程即为先序遍历这棵状态树的过程，如算法6.14所示：

```
void PowerSet(int i, int n){  
    if (i>n)  输出幂集的一个元素;  
    else{  取第i个元素: PowerSet(i+1, n);  
        舍第i个元素: PowerSet(i+1, n);  
    }  
}
```

- 图6.28所示的状态变化树是一棵满二叉树，树中每个结点都对应一个解。

6.7 回溯法与树的遍历

- 然而在很多问题背景下，每个叶子结点不一定对应一个解。这类问题可以看成是在约束条件下进行先序遍历，并在遍历过程中剪去那些不满足条件的分支。
- 例如4皇后问题，如图6.29。
 - 根结点表示棋盘的初始状态。
 - 每个皇后棋子都有4个可选择的位置，但在任何时刻，棋盘的合法布局都必须满足3个约束条件：任何两个皇后都不占据棋盘上的同一行、同一列、同一对角线。
- 求所有合法布局的过程即为在上述约束条件下先序遍历图6.29的状态树的过程。这个过程，可作为回溯法的一般模式。

6.7 回溯法与树的遍历

- 遍历中访问结点的操作为，
 - 判别棋盘上是否已经有一个完整的布局，
 - 若是则输出该布局；
 - 否则依次先序遍历满足约束条件的各棵子树，即首先判断该子树根的布局是否合法，若合法，则先序遍历该子树，否则剪去该子树分支。

```
void Trial (int i, int n){  
{  
    if (i>n)  输出棋盘的当前布局;  
    else for (j=1; j<=n; j++){  
        在第i行第j列放置一个棋子;  
        if (当前布局合法) Trial(i+1, n);  
        移走第i行第j列的棋子;  
    }  
}
```



补充:

遍历二叉树的三种方法:

1. 递归方法
2. 使用栈
3. 不用栈的二叉树遍历的非递归方法

补充:

1、递归方法

- 当给出二叉树的链式存储结构以后，用具有递归功能的程序设计语言很方便就能实现上述算法。然而，并非所有程序设计语言都允许递归；另一方面，递归程序虽然简洁，但可读性一般不好，执行效率也不高。

补充:

2、使用栈

- 中序非递归遍历算法如书P130
- 先序非递归遍历算法与此类似
- 后序非递归遍历算法比较复杂一点
 - 后序遍历与先序遍历和中序遍历不同，在后序遍历过程中，结点在第一次出栈后，还需再次入栈，也就是说，**结点要入两次栈，出两次栈**，而访问结点是在第二次出栈时访问。因此，为了区别同一个结点指针的两次出栈，要在结点结构中设置一标志tag
 - 当tag==L时，表示第一次出栈，这时不能访问，
 - 当tag==R时，表示第二次出栈，这时可以访问。



```
#define maxsize 100
typedef enum{L,R} tagtype;
typedef struct
{
    BiTree ptr;
    tagtype tag;
}Stacknode;

typedef struct
{
    Stacknode Elem[maxsize];
    int top;
}SqStack;
```

```
void PostOrderUnrec(BiTree t)
```

```
{  
    SqStack s; Stacknode x; InitStack (s); p=t;  
    do{  
        while (p!=null) { //遍历左子树  
            x.ptr = p;  
            x.tag = L;    //标记为左子树  
            push(s,x);  
            p=p->lchild;  
        }  
        while (!StackEmpty(s) && s.Elem[s.top].tag==R){  
            x = pop(s);  
            p = x.ptr;  
            visit(p->data); //tag为R, 表示右子树访问完毕, 故访问根结点  
        }  
        if (!StackEmpty(s)) {  
            s.Elem[s.top].tag =R; //遍历右子树  
            p=s.Elem[s.top].ptr->rchild;  
        }  
    }while (!StackEmpty(s));  
} //PostOrderUnrec
```

补充:

3、不用栈的二叉树遍历的非递归方法

- 前面介绍的二叉树的遍历算法可分为两类:一类是依据二叉树结构的递归性,采用递归调用的方式来实现;另一类则是通过堆栈或队列来辅助实现。
- 采用这两类方法对二叉树进行遍历时,递归调用和栈的使用都带来额外空间增加,递归调用的深度和栈的大小是动态变化的,都与二叉树的高度有关。因此,在最坏的情况下,即二叉树退化为单支树的情况下,递归的深度或栈需要的存储空间等于二叉树中的结点数。
- 还有一类二叉树的遍历算法,就是不用栈也不用递归来实现。

■ 常用的不用栈的二叉树遍历的非递归方法有以下三种：

(1) **对二叉树采用三叉链表存放**，即在二叉树的每个结点中增加一个双亲域parent，这样，在遍历深入到不能再深入时，可沿着走过的路径回退到任何一棵子树的根结点，并再向另一方向走。由于这一方法的实现是在每个结点的存储上又增加一个双亲域，故其存储开销就会增加。

■ 如赫夫曼树的编码算法6.13

(2) 采用逆转链的方法，不需要附加的栈空间，而是在周游的过程中当沿着结点的左或右子树“下降”时，临时改变结点的lchild或rchild的值，使之指向结点的双亲，从而为以后的“上升”提供路径；上升的过程中将结点lchild和rchild恢复为原来的值。此算法要求二叉树用lchild-rchild法存储，每个结点再附加一个一位的tag字段，用来表示是否进入该结点的左子树。（具体算法参看许卓群书。）

(3) 在线索二叉树上的遍历，即利用具有 n 个结点的二叉树中的叶子结点和一度结点的 $n+1$ 个空指针域，来存放线索，然后在这种具有线索的二叉树上遍历时，就可不需要栈，也不需要递归了。如算法6.5

本章小结

1. 熟练掌握二叉树的结构特性，了解相应的证明方法。
2. 熟悉二叉树的各种存储结构的特点及适用范围。
3. 遍历二叉树是二叉树各种操作的基础。实现二叉树遍历的具体算法与所采用的存储结构有关。掌握各种遍历策略的递归算法，灵活运用遍历算法实现二叉树的其它操作。层次遍历是按另一种搜索策略进行的遍历。
4. 理解二叉树线索化的实质是建立结点与其在相应序列中的前驱或后继之间的直接联系，熟练掌握二叉树的线索化过程以及在中序线索化树上找给定结点的前驱和后继的方法。二叉树的线索化过程是基于对二叉树进行遍历，而线索二叉树上的线索又为相应的遍历提供了方便。

本章小结

5. 熟悉树的各种存储结构及其特点，掌握树和森林与二叉树的转换方法。建立存储结构是进行其它操作的前提，因此读者应掌握 1 至 2 种建立二叉树和树的存储结构的方法。
6. 学会编写实现树的各种操作的算法。
7. 了解最优树的特性，掌握建立最优树和哈夫曼编码的方法。



■ 习题

■ 6.3, 6.5, 6.7, 6.33

■ 6.12, 6.13, 6.20, 6.37, 6.38, 6.43, 6.56

■ 6.21, 6.26, 6.27, 6.29, 6.62